

Lecture 16

Generative Models: Diffusion and Flows

Speaker: Kaiming He

Overview

- Diffusion Models
- Flow Matching

Diffusion Models

Diffusion: Generative Models that Work

Text-to-image generation



Generated by Stable Diffusion 3 Medium.

Prompt: teddy bear teaching a course, with "generative models" written on blackboard

Diffusion: Generative Models that Work

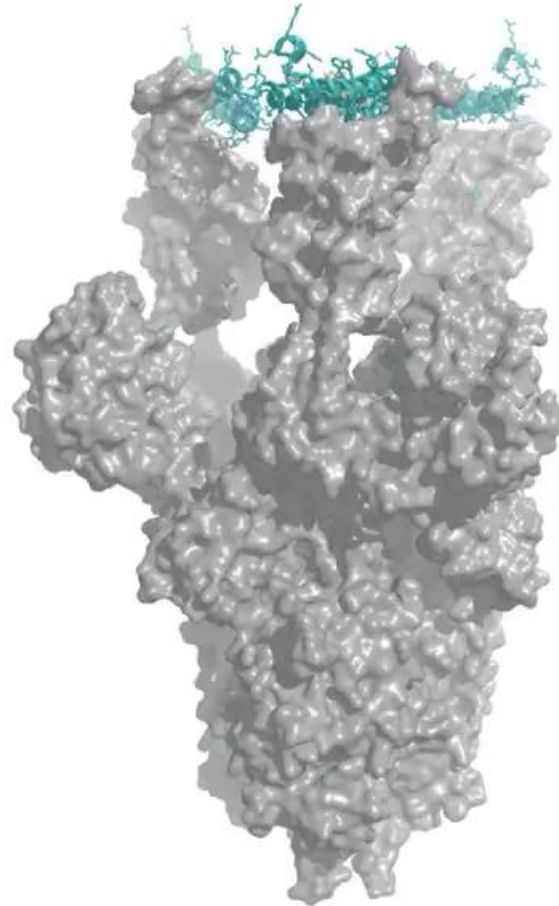
Text-to-video generation



Generated by Sora

Diffusion: Generative Models that Work

Protein design and generation



The original paper on diffusion models

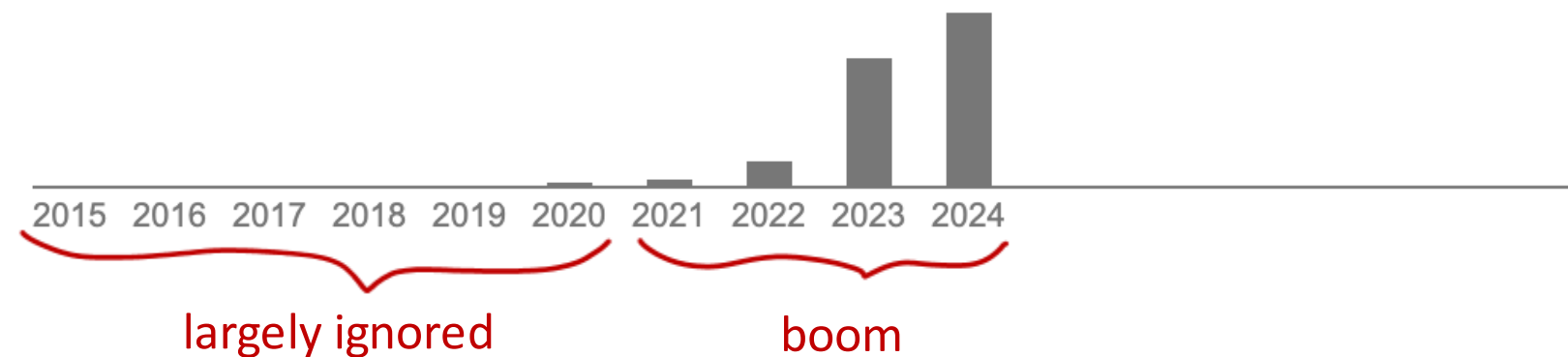
Deep unsupervised learning using nonequilibrium thermodynamics

Authors Jascha Sohl-Dickstein, Eric A Weiss, Niru Maheswaranathan, Surya Ganguli

Publication date 2015/3/12

Journal International Conference on Machine Learning

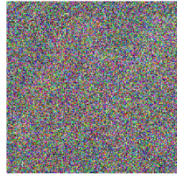
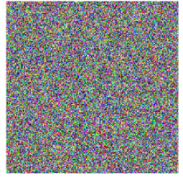
Total citations Cited by 5630



Diffusion Models in a nutshell

noise

data



x_T

...

x_t

x_{t-1}

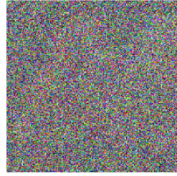
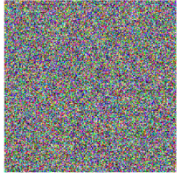
...

x_0



Diffusion Models in a nutshell

noise



data

x_T



...



x_t



x_{t-1}



...

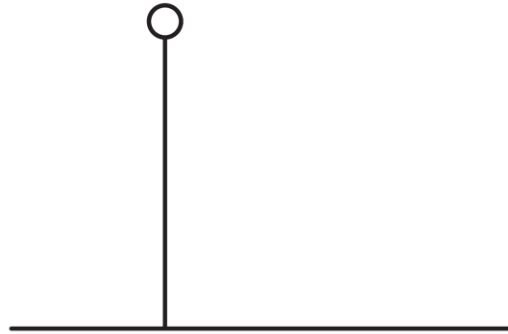


x_0

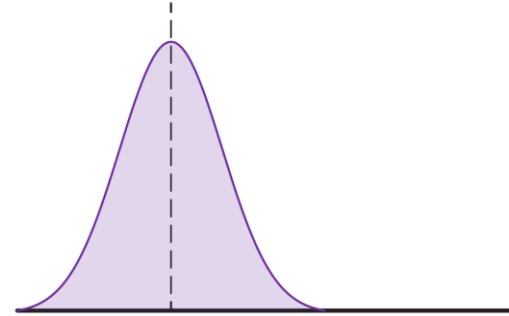
Reverse process: denoise

What is noise?

- Adding Gaussian noise $\Leftrightarrow$ sampling $x \sim \mathcal{N}(x \mid x_0, \sigma)$



$$p(x) = \delta(x - x_0)$$

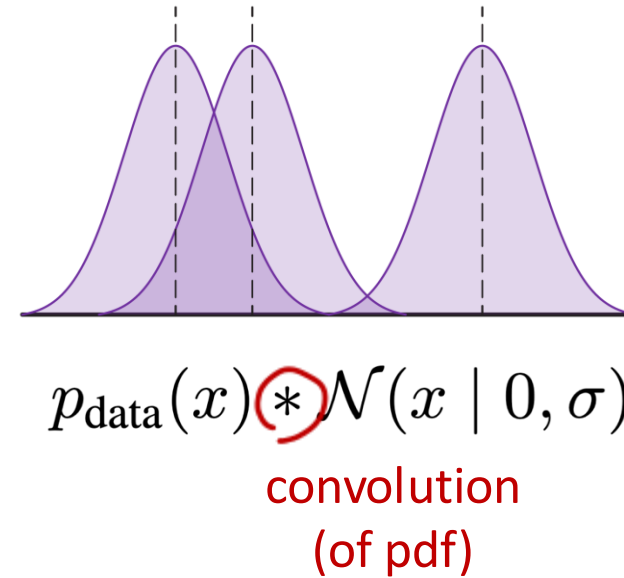
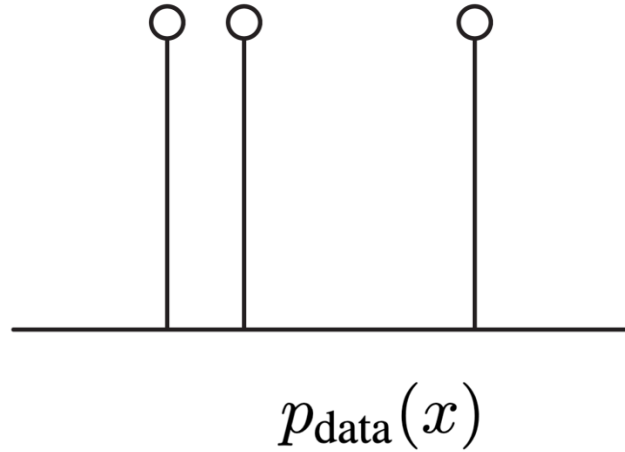


$$p(x) = \mathcal{N}(x \mid x_0, \sigma)$$



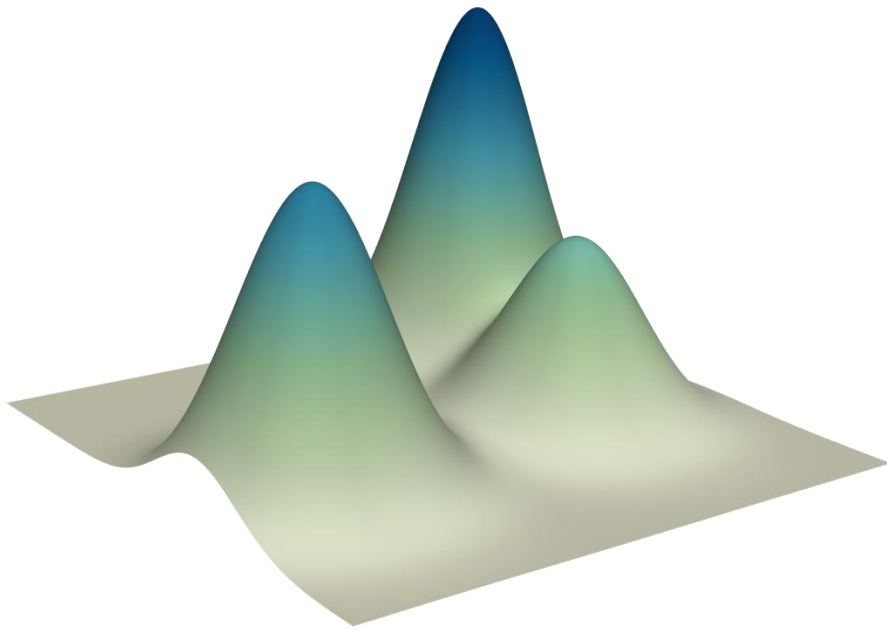
What is noise?

- Adding Gaussian noise $\Leftrightarrow$ sampling $x \sim \mathcal{N}(x \mid x_0, \sigma)$

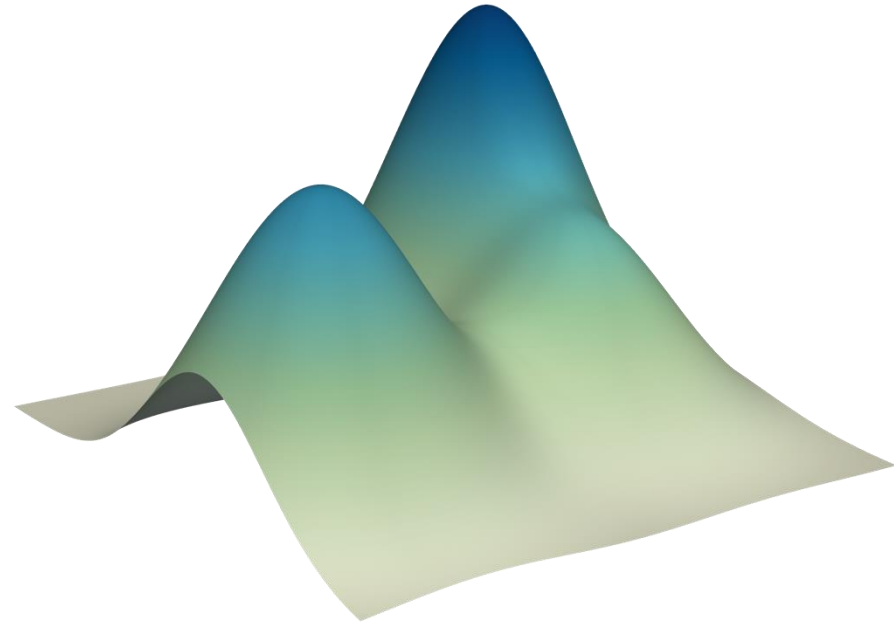


What is noise?

- Adding Gaussian noise $\Leftrightarrow$ sampling $x \sim \mathcal{N}(x \mid x_0, \sigma)$



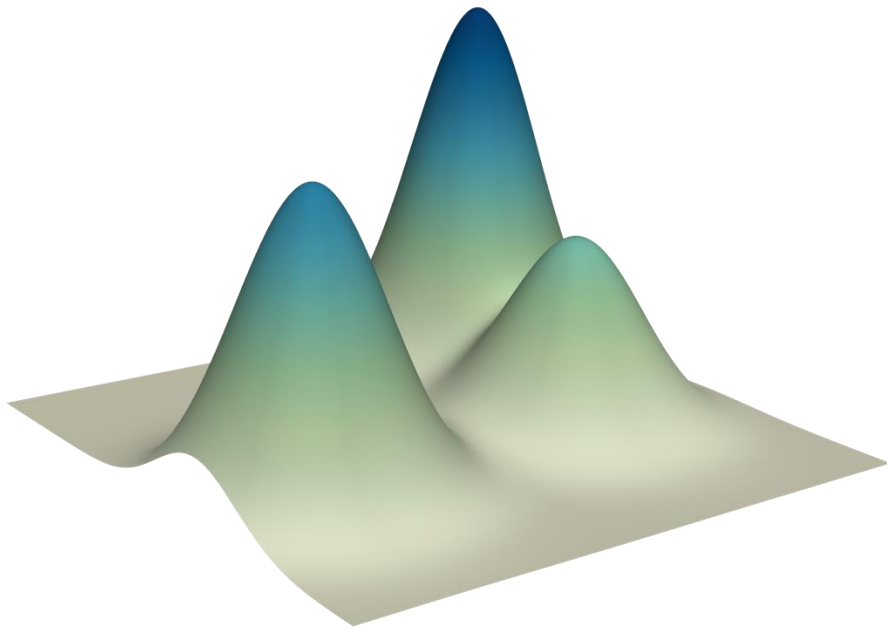
$p_{\text{data}}(x)$



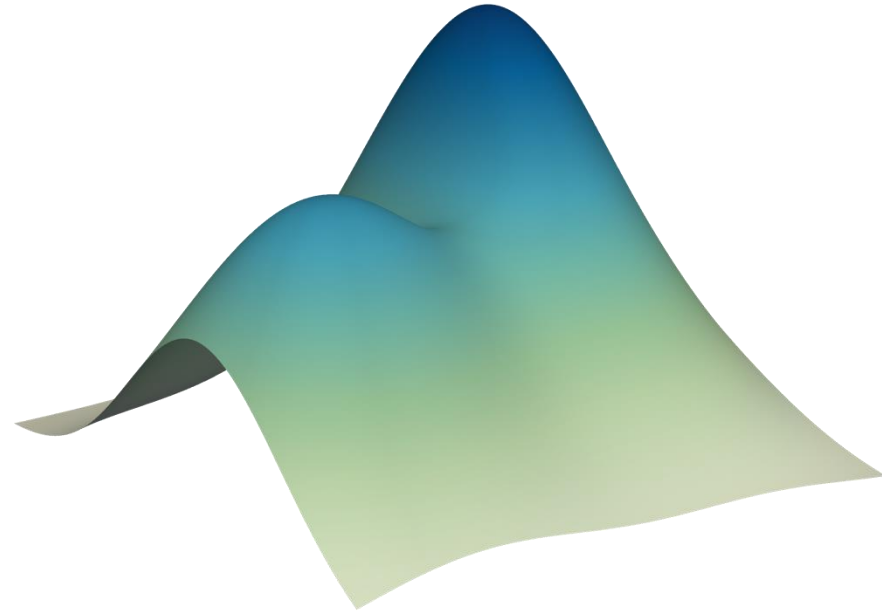
$p_{\text{data}}(x) * \mathcal{N}(x \mid 0, \sigma)$

What is noise?

- Adding Gaussian noise $\Leftrightarrow$ sampling $x \sim \mathcal{N}(x \mid x_0, \sigma)$



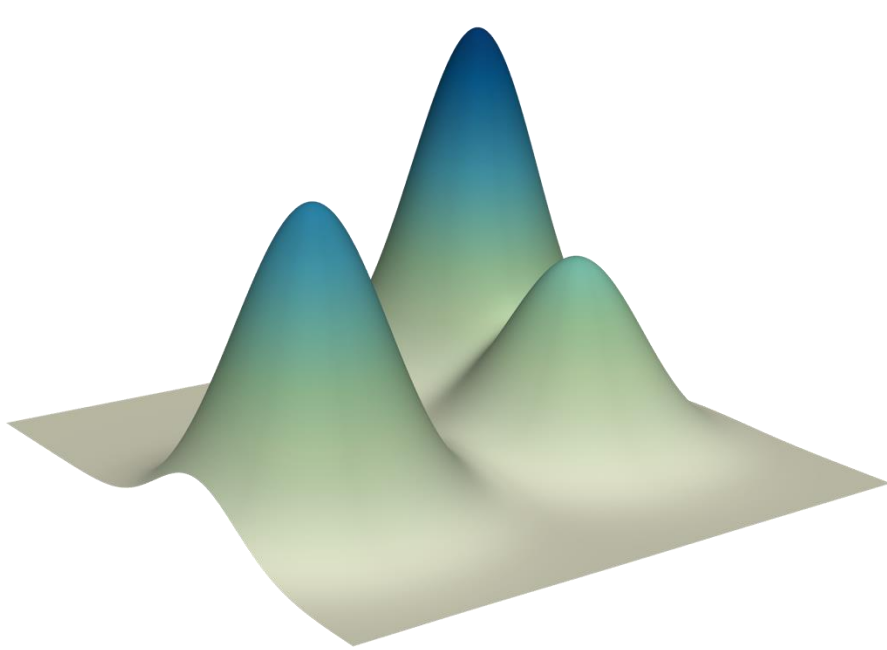
$$p_{\text{data}}(x)$$



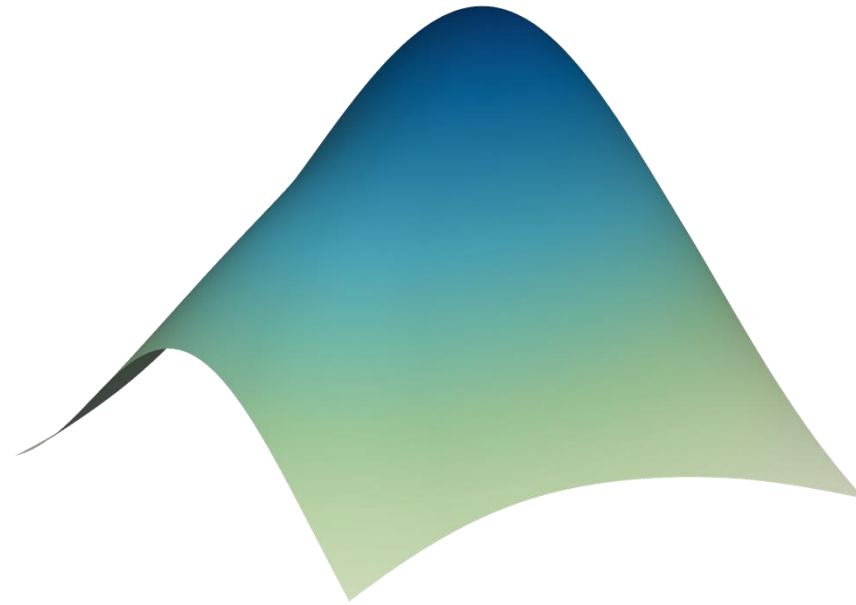
$$p_{\text{data}}(x) * \mathcal{N}(x \mid 0, \sigma)$$

What is noise?

- Adding Gaussian noise $\Leftrightarrow$ sampling $x \sim \mathcal{N}(x \mid x_0, \sigma)$



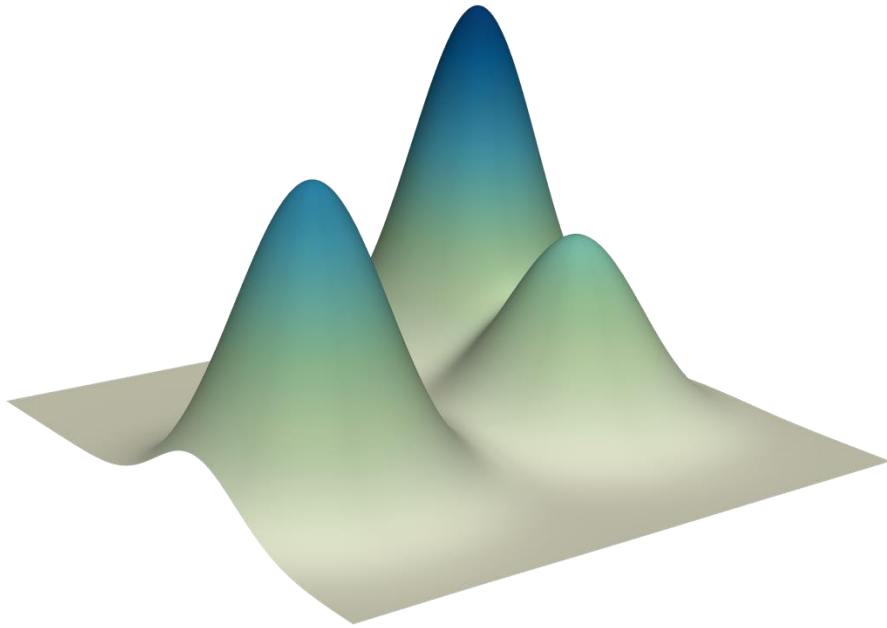
$$p_{\text{data}}(x)$$



$$p_{\text{data}}(x) * \mathcal{N}(x \mid 0, \sigma)$$

What is noise?

- Adding Gaussian noise $\Leftrightarrow$ sampling $x \sim \mathcal{N}(x \mid x_0, \sigma)$

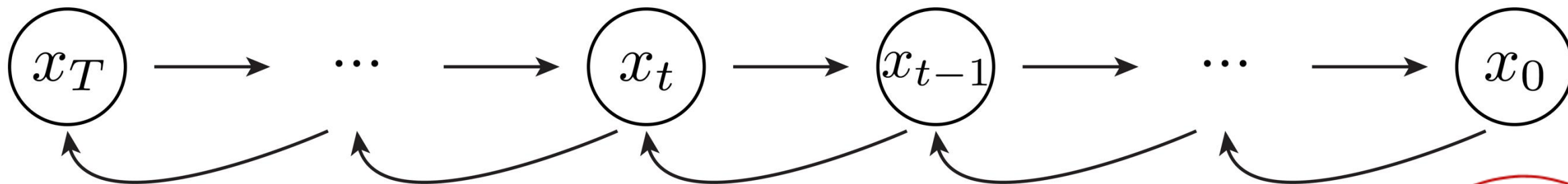
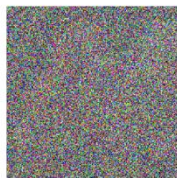
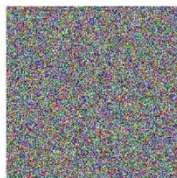


$$p_{\text{data}}(x)$$

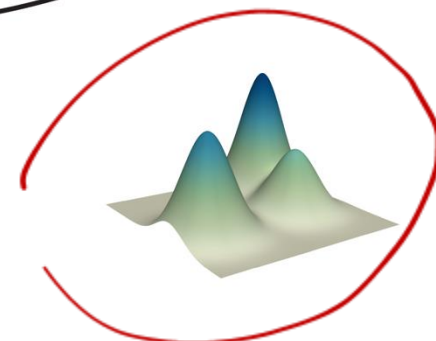
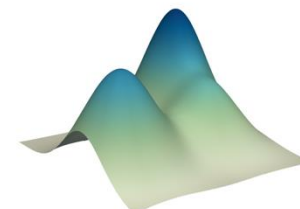
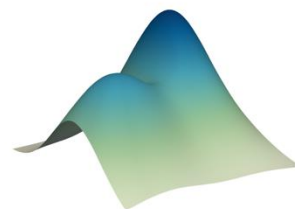


$$p_{\text{data}}(x) * \mathcal{N}(x \mid 0, \sigma)$$

What is noise?



noise
distribution



data
distribution

Diffusion Models

Forward process

- add noise to data

Reverse process

- learn to denoise

Training objective

- from Hierarchical VAE to L2 loss

Noise Conditional Network

- represent distributions

Diffusion Models

Forward process

- add noise to data

Reverse process

- learn to denoise

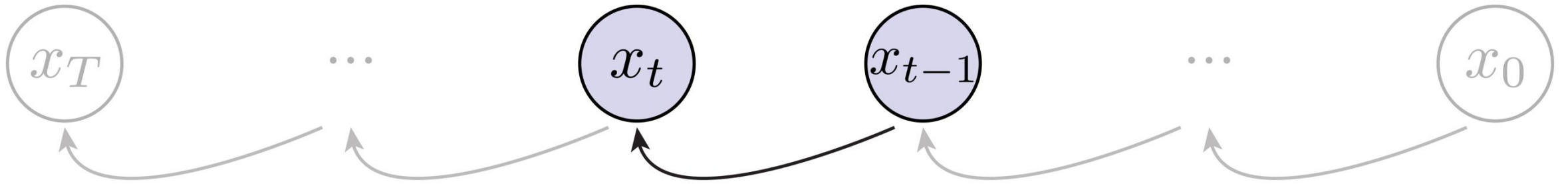
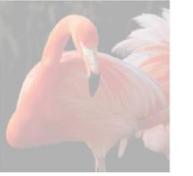
Training objective

- from Hierarchical VAE to L2 loss

Noise Conditional Network

- represent distributions

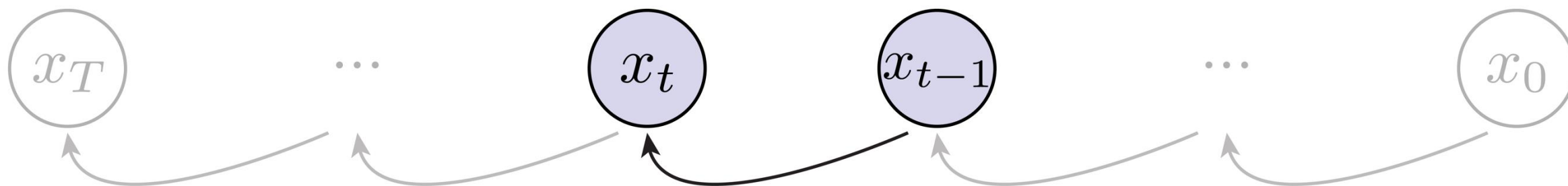
Forward Process



$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

coefficients:
variance preserving

Forward Process

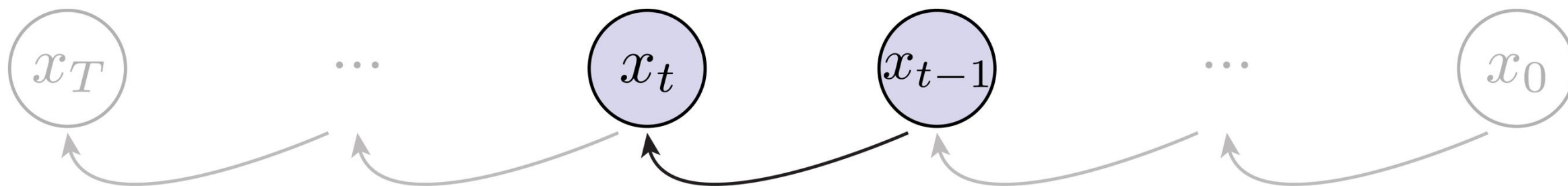


$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

t: schedule

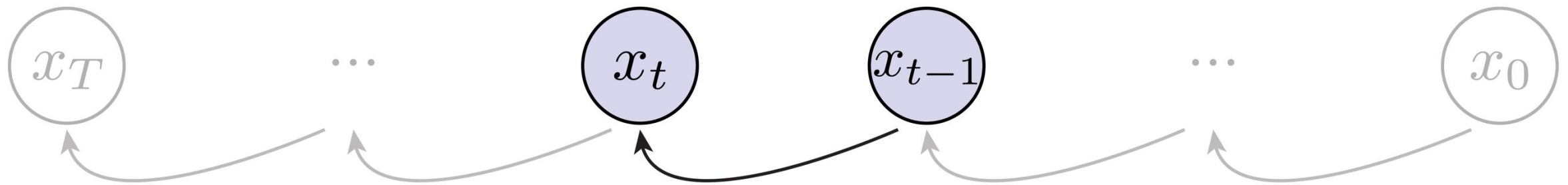
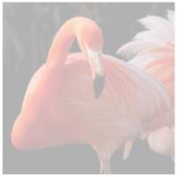
key to Diffusion Models' success

Forward Process



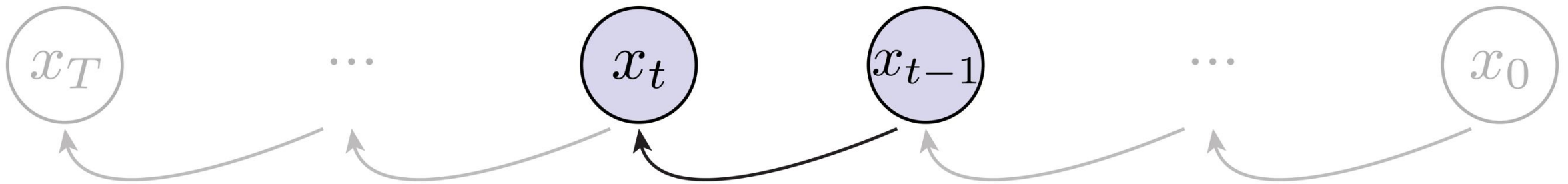
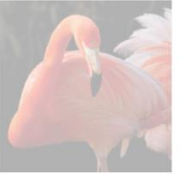
$$x_t = \underbrace{\sqrt{1 - \beta_t}}_{\text{mean of } x_t} x_{t-1} + \underbrace{\sqrt{\beta_t}}_{\text{std of } x_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I})$$

Forward Process



$$q(x_t | x_{t-1})$$
$$= \mathcal{N}(x_t | \underbrace{\sqrt{1 - \beta_t} x_{t-1}}_{\text{mean of } x_t}, \underbrace{\beta_t \mathbf{I}}_{\text{var of } x_t})$$

Forward Process

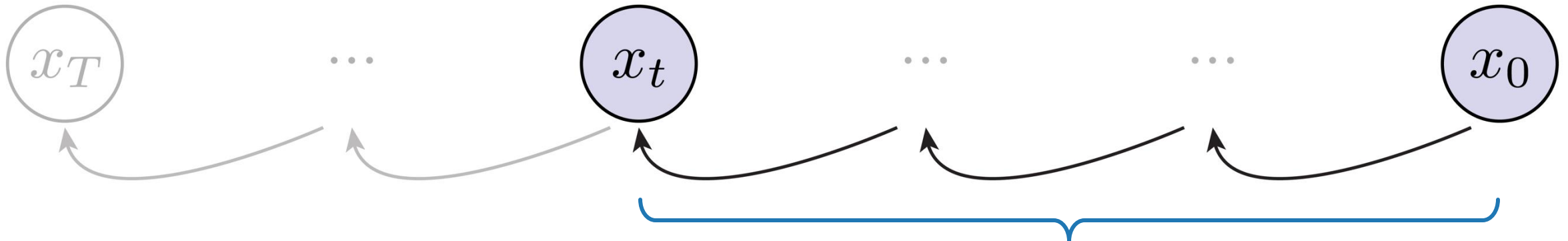


$$q(x_t \mid x_{t-1})$$
$$= \mathcal{N}(x_t \mid \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I})$$

identity matrix

- sampling is i.i.d.
- dim = dim of data

Forward Process



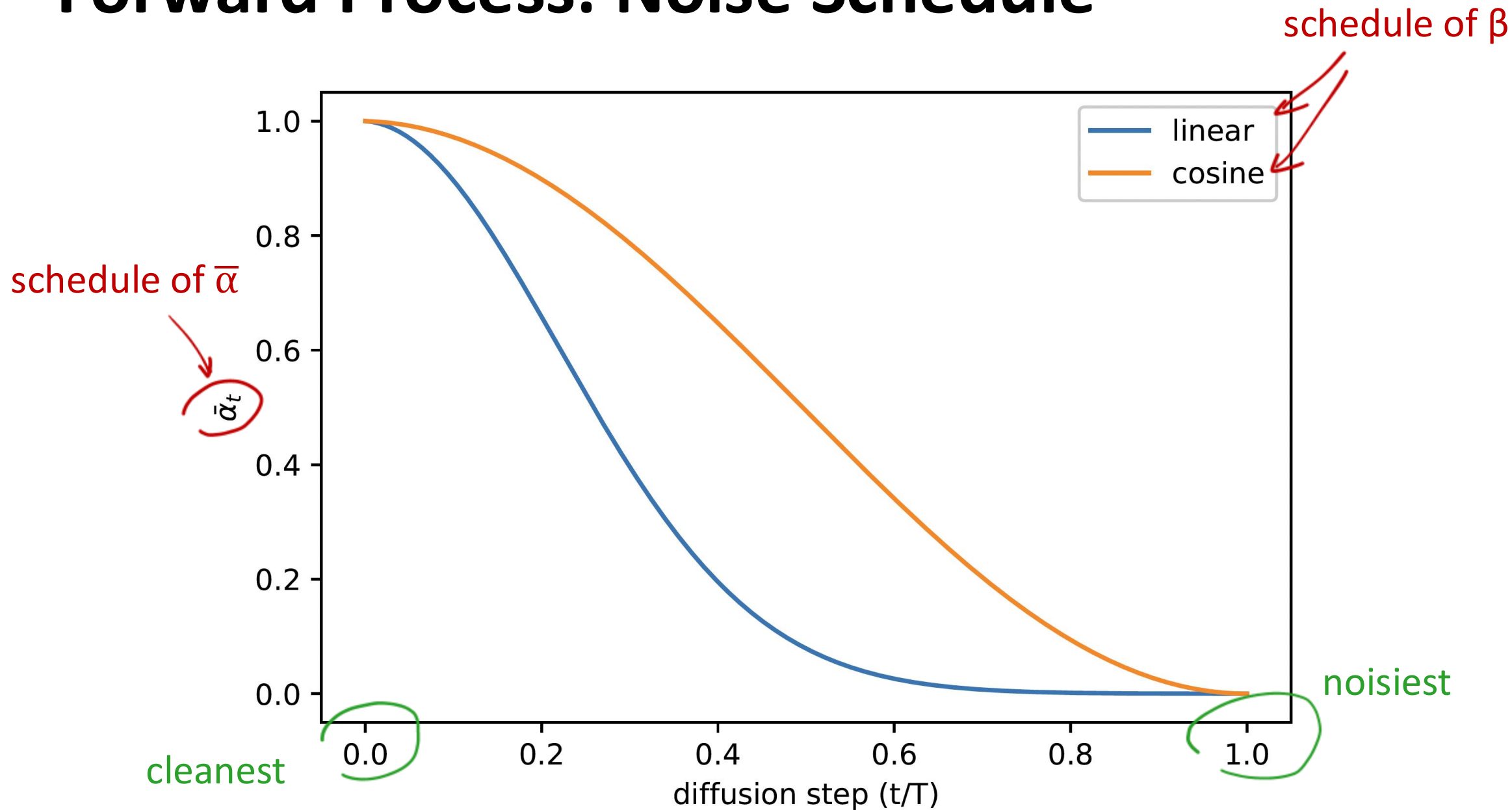
- sampling without iterating
- x_t from x_0 in closed form

$$q(x_t | x_0) = \mathcal{N}(x_t | \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

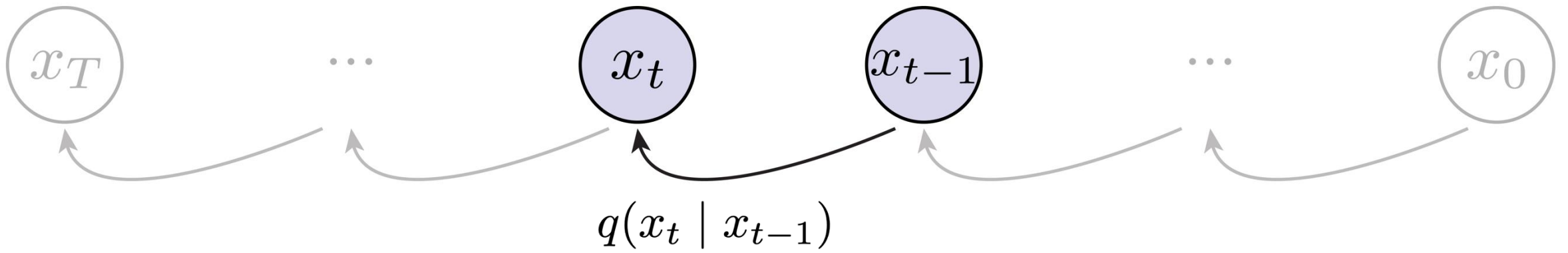
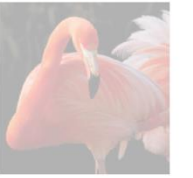
schedule
derived from β

$$\alpha_t := 1 - \beta_t$$
$$\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$$

Forward Process: Noise Schedule



Forward Process



tl; dr:

- pre-defined conditional distributions
- Gaussian w/ controllable mean/std
- easy to sample from

Diffusion Models

Forward process

- add noise to data

Reverse process

- learn to denoise

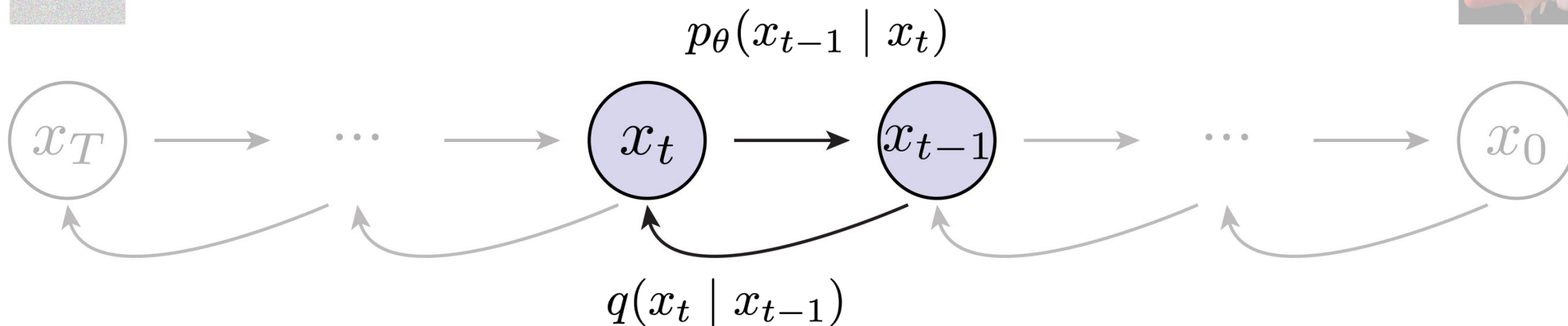
Training objective

- from Hierarchical VAE to L2 loss

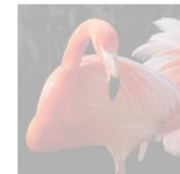
Noise Conditional Network

- represent distributions

Reverse Process



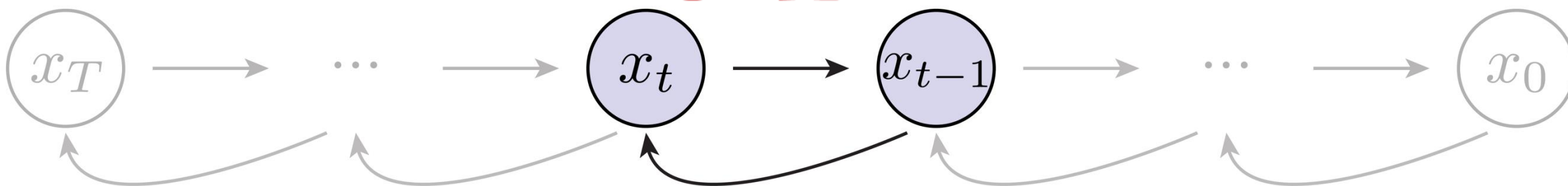
Reverse Process



parameterized
by a network

$$p_{\theta}(\underline{x_{t-1}} \mid \underline{x_t})$$

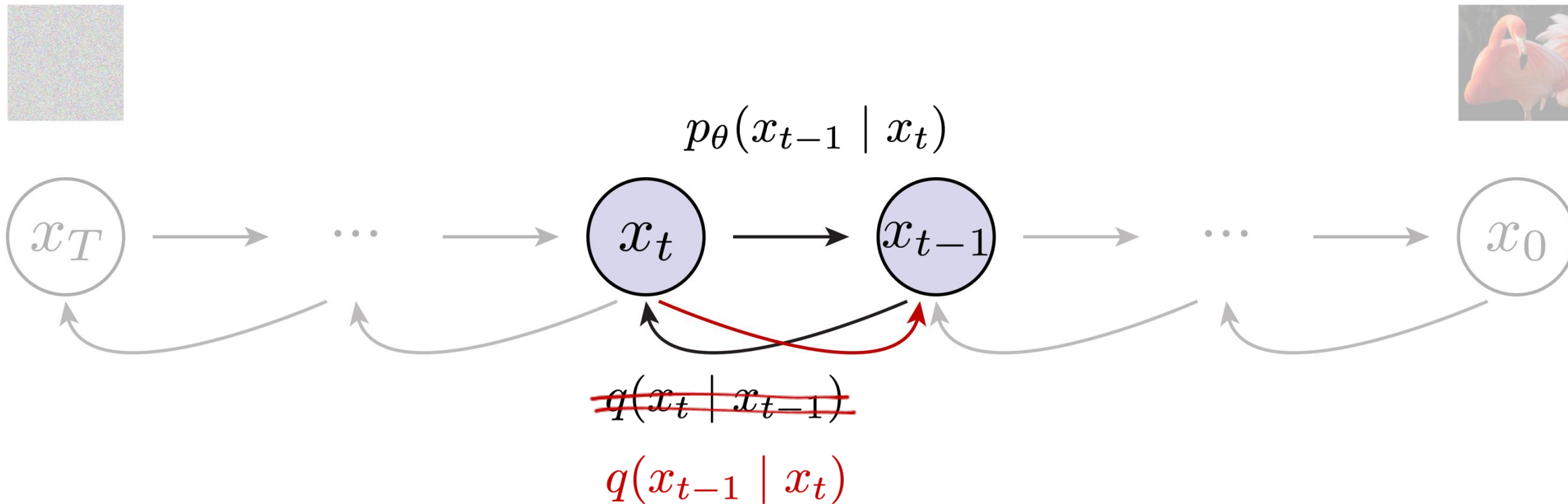
reverse the
time steps



$$\underline{q(x_t \mid x_{t-1})}$$

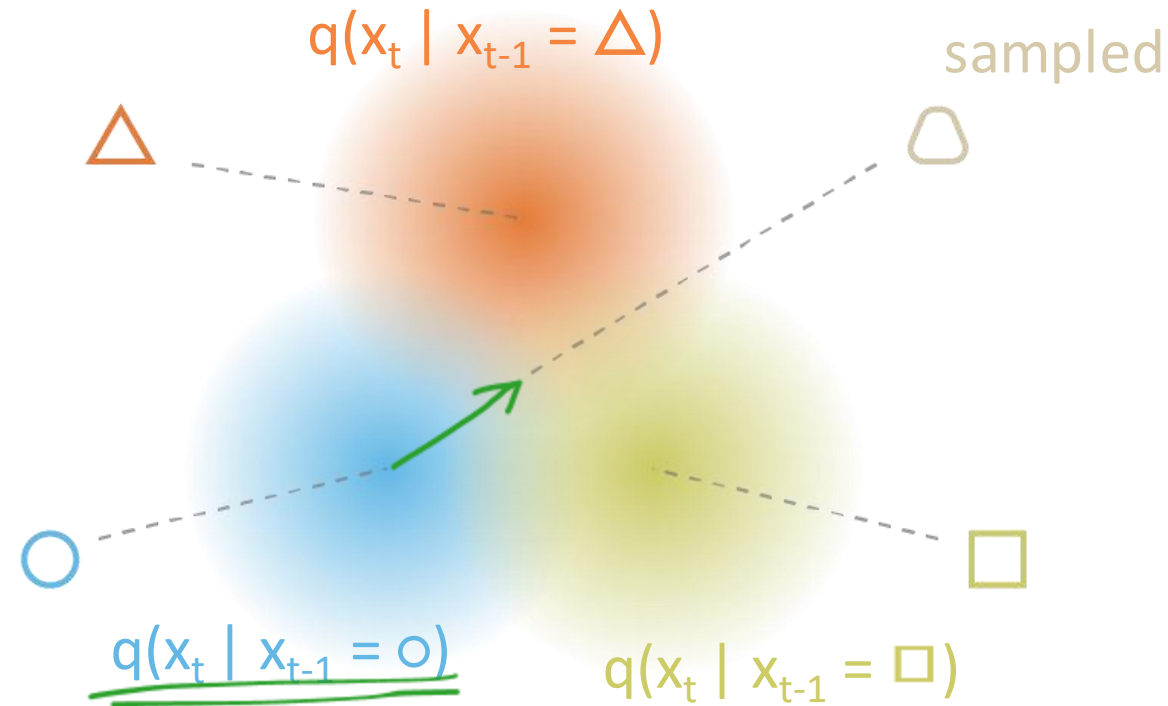
- known
- but not our target

Reverse Process



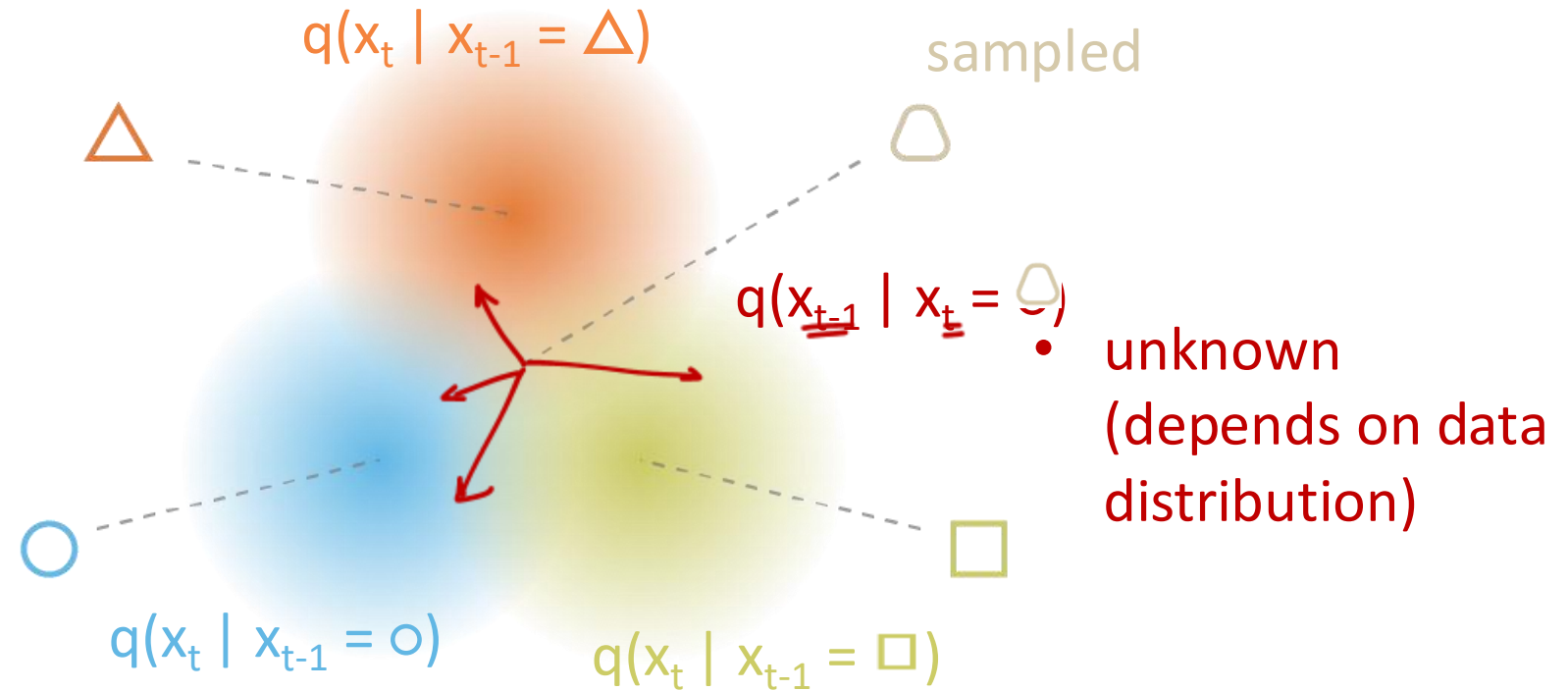
- our target
- but unknown

Why are the reverse conditionals unknown?



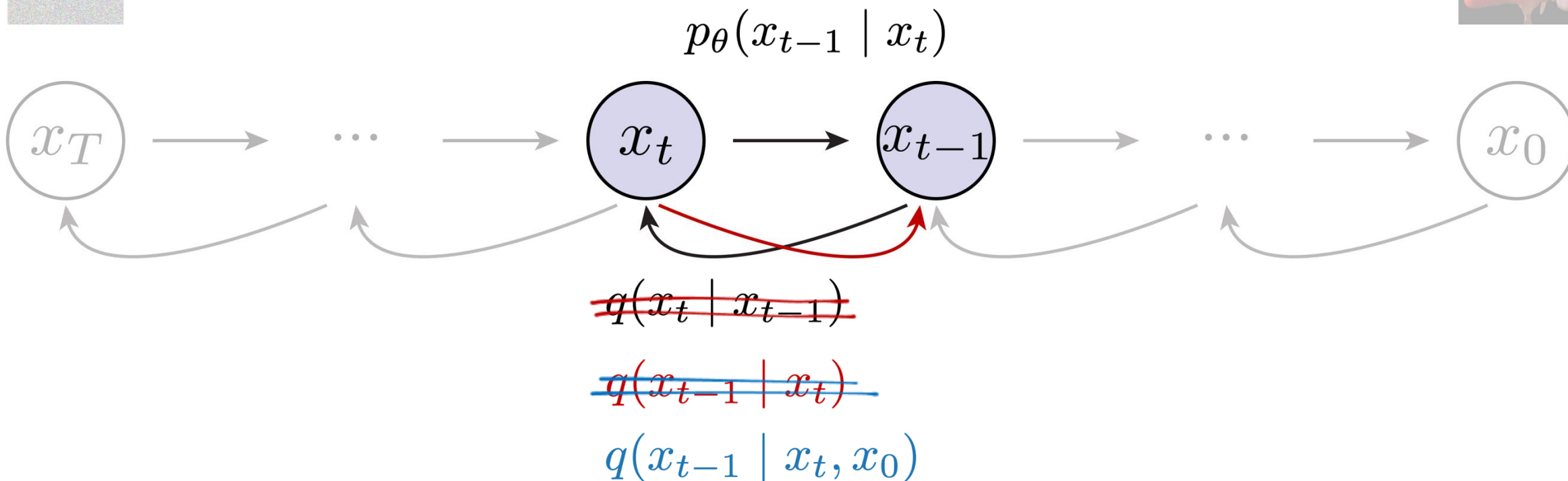
- known (Gaussian)

Why are the reverse conditionals unknown?



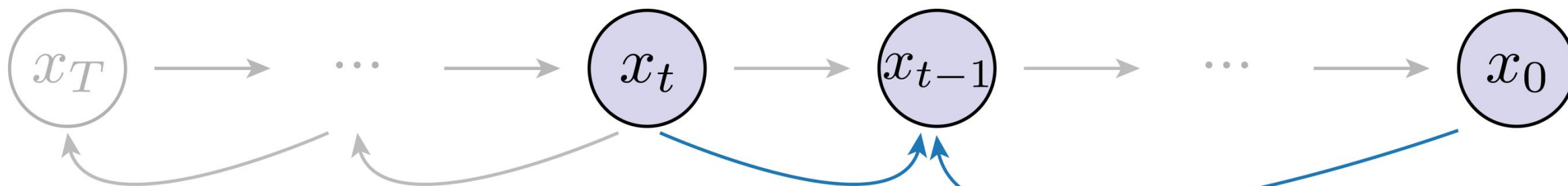
Reverse Process

- The formulation of “x-conditional” then “**marginalization**” is a common strategy (recap from our lecture on VAE)



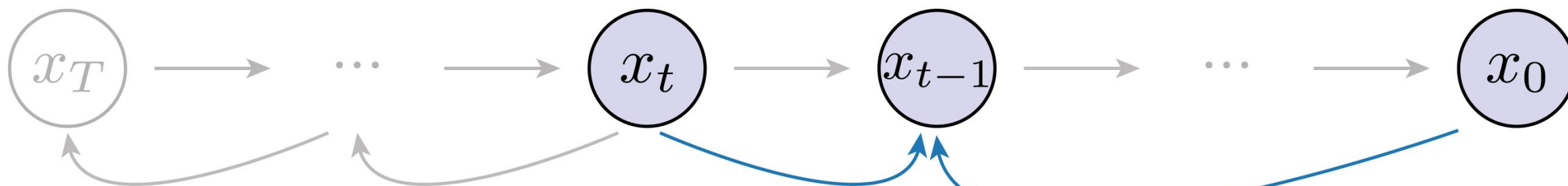
- known
- Gaussian

Reverse Process



$$q(x_{t-1} \mid x_t, x_0)$$
$$= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

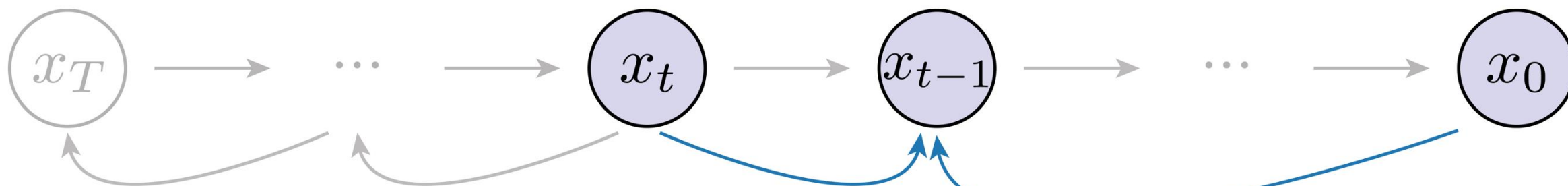
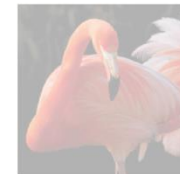
Reverse Process



$$\begin{aligned} & q(x_{t-1} \mid x_t, x_0) \\ &= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I}) \end{aligned}$$

$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$
var

Reverse Process



$$q(x_{t-1} \mid x_t, x_0)$$

$$= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

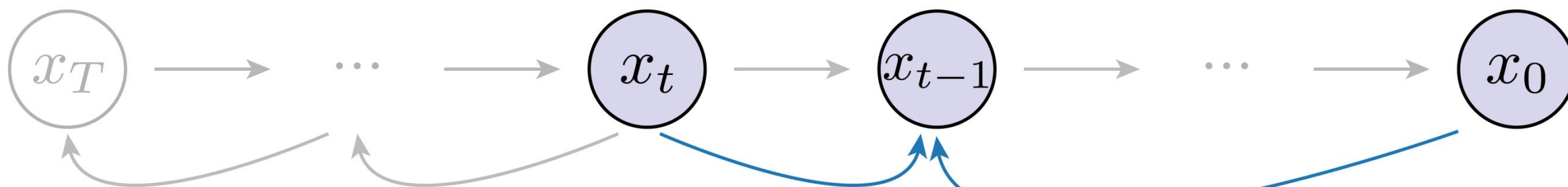
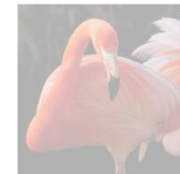
$$\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t$$

mean

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t$$

var

Reverse Process



$$q(x_{t-1} \mid x_t, x_0)$$

$$= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

$$\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t}x_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t$$

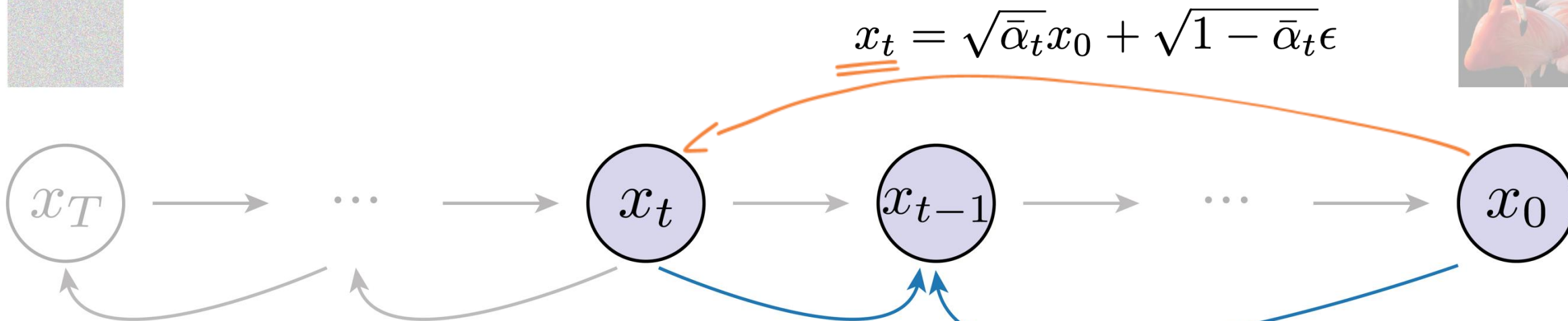
mean

linear combination

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

var

Reverse Process



$$q(x_{t-1} \mid x_t, x_0)$$

$$= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

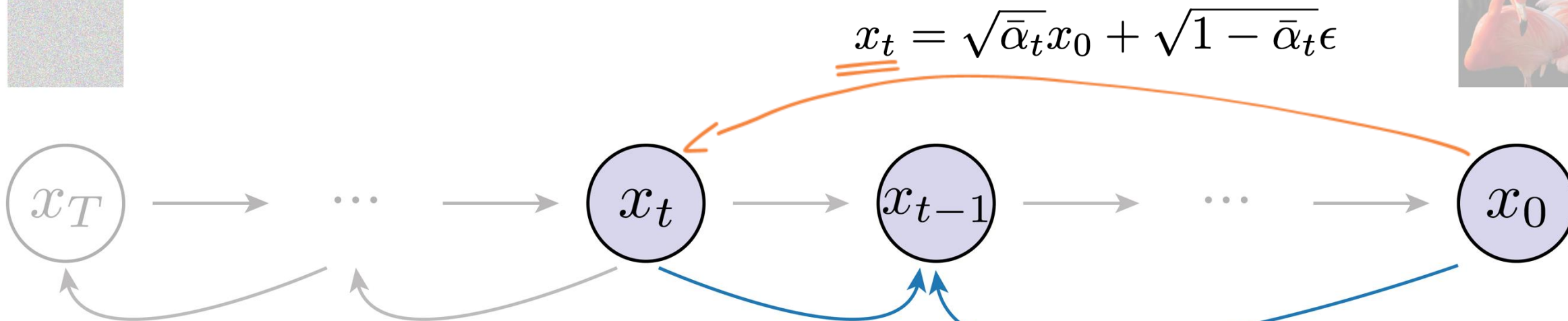
$$\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_t} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

mean

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

var

Reverse Process



$$q(x_{t-1} \mid x_t, x_0)$$

$$= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

$$\tilde{\mu}_t(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_t} \beta_t}{1 - \alpha_t} x_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

mean

$$= \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right) \quad \text{"noise"}$$

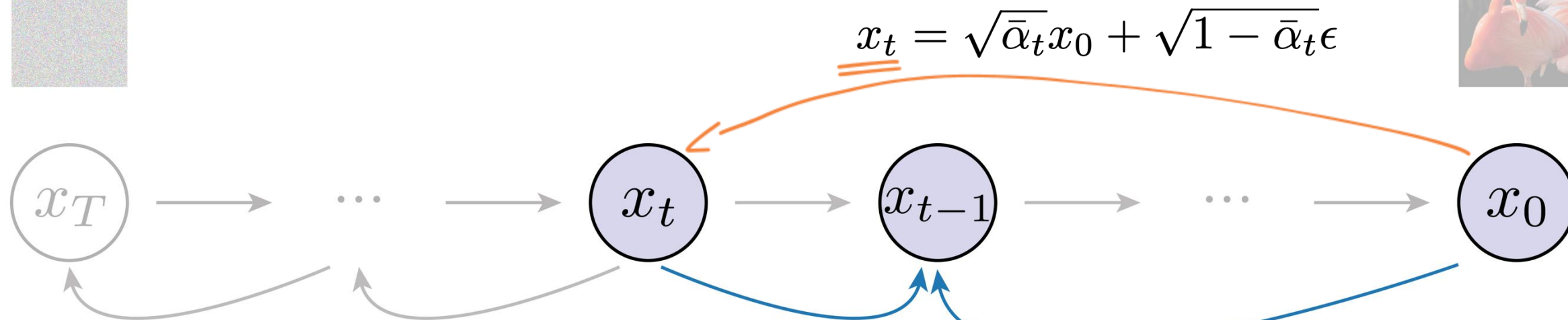
$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

var

Reverse Process

tl; dr:

- outcome of the dependency graph
- some linear combinations



$$q(x_{t-1} \mid x_t, x_0)$$

$$= \mathcal{N}(x_{t-1} \mid \tilde{\mu}(x_t, x_0), \tilde{\beta}_t \mathbf{I})$$

$$\tilde{\mu}(x_t, x_0) := \frac{\sqrt{\bar{\alpha}_t} \beta_t}{1 - \bar{\alpha}_t} x_0 + \frac{\sqrt{\bar{\alpha}_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t$$

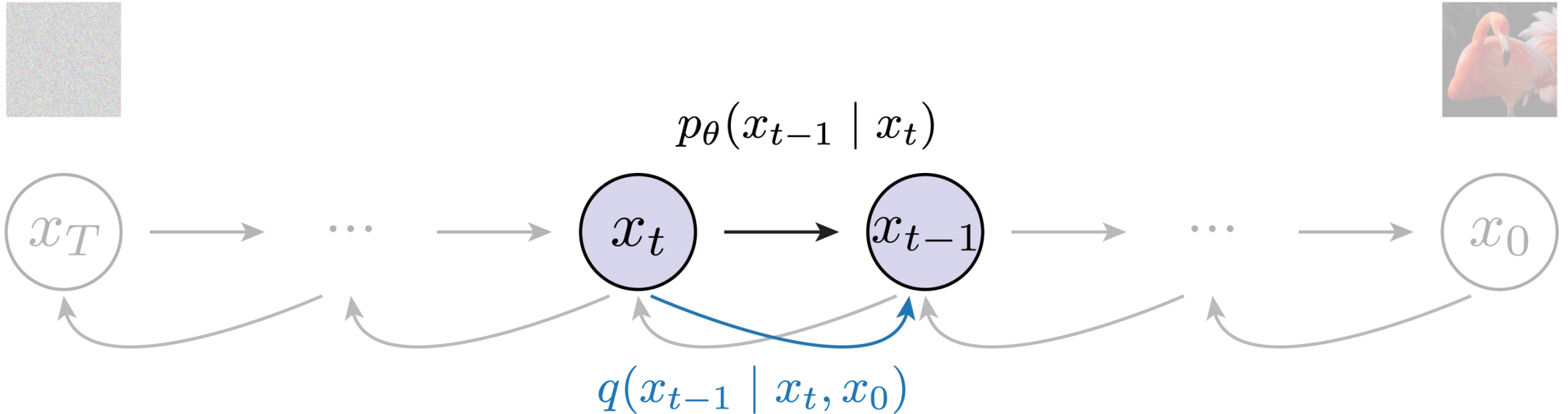
mean

$$= \frac{1}{\sqrt{\bar{\alpha}_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right) \quad \text{"noise"}$$

$$\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$$

var

Reverse Process

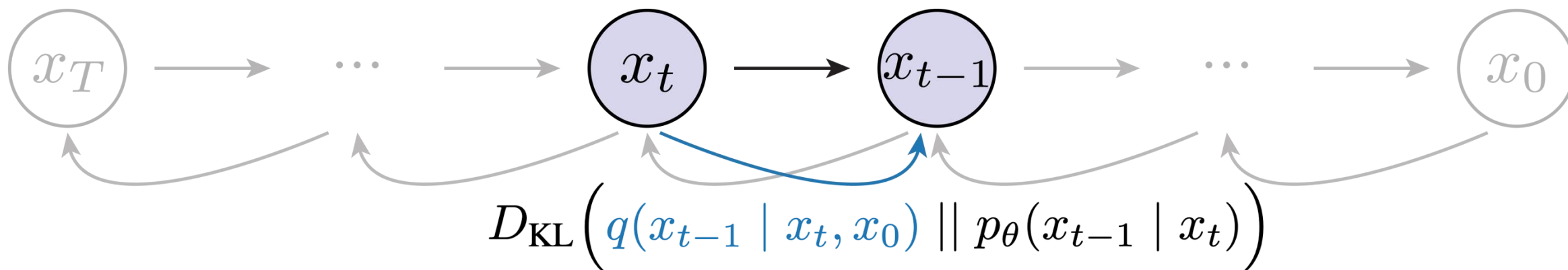
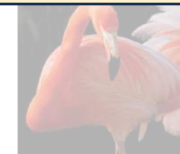


- **tl; dr:** a known Gaussian
- we want to learn it by p_θ
- we can represent p_θ by a Gaussian
- minimize KL divergence

Reverse Process

D_{KL} of two Gaussians is like L2 loss:

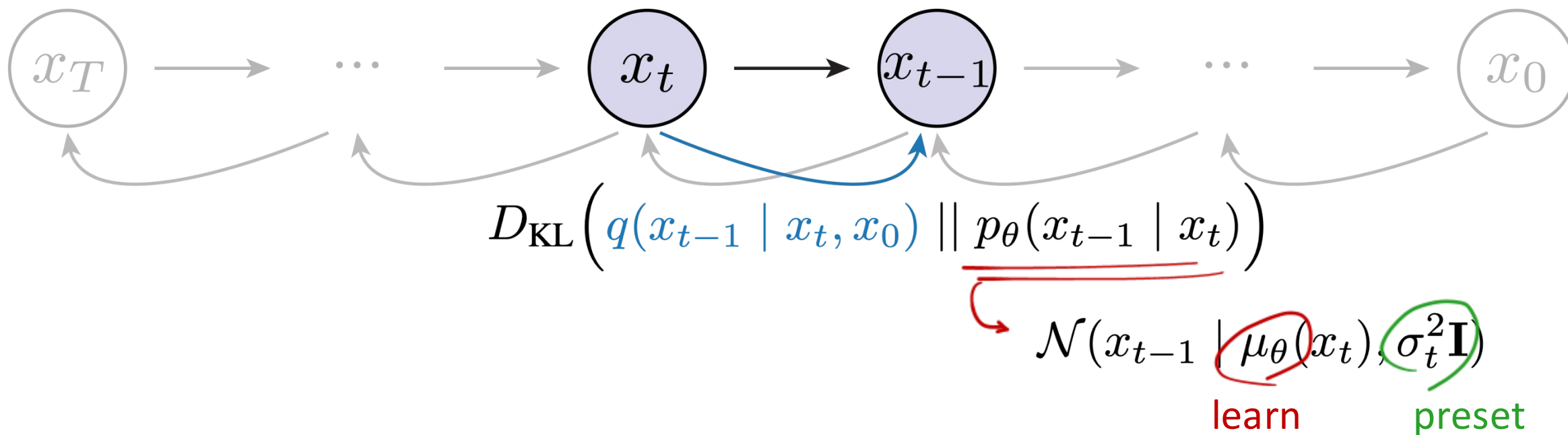
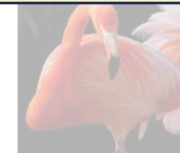
$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$



Reverse Process

D_{KL} of two Gaussians is like L2 loss: (pset 1)

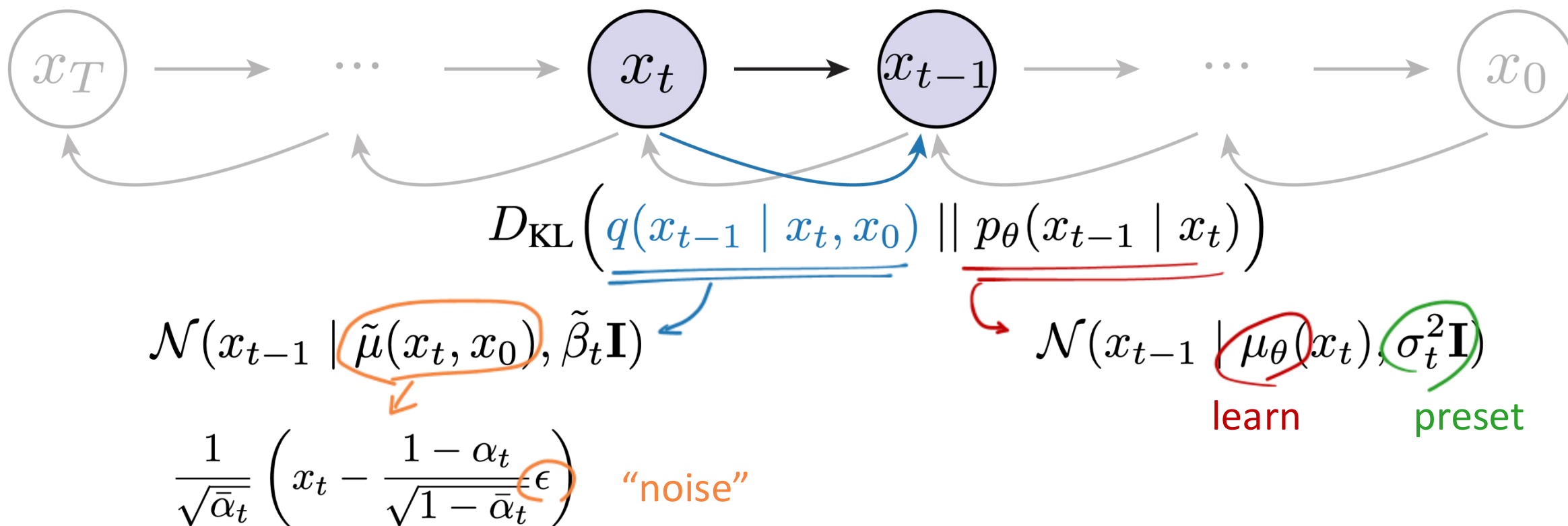
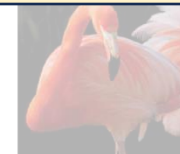
$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$



Reverse Process

D_{KL} of two Gaussians is like L2 loss: (pset 1)

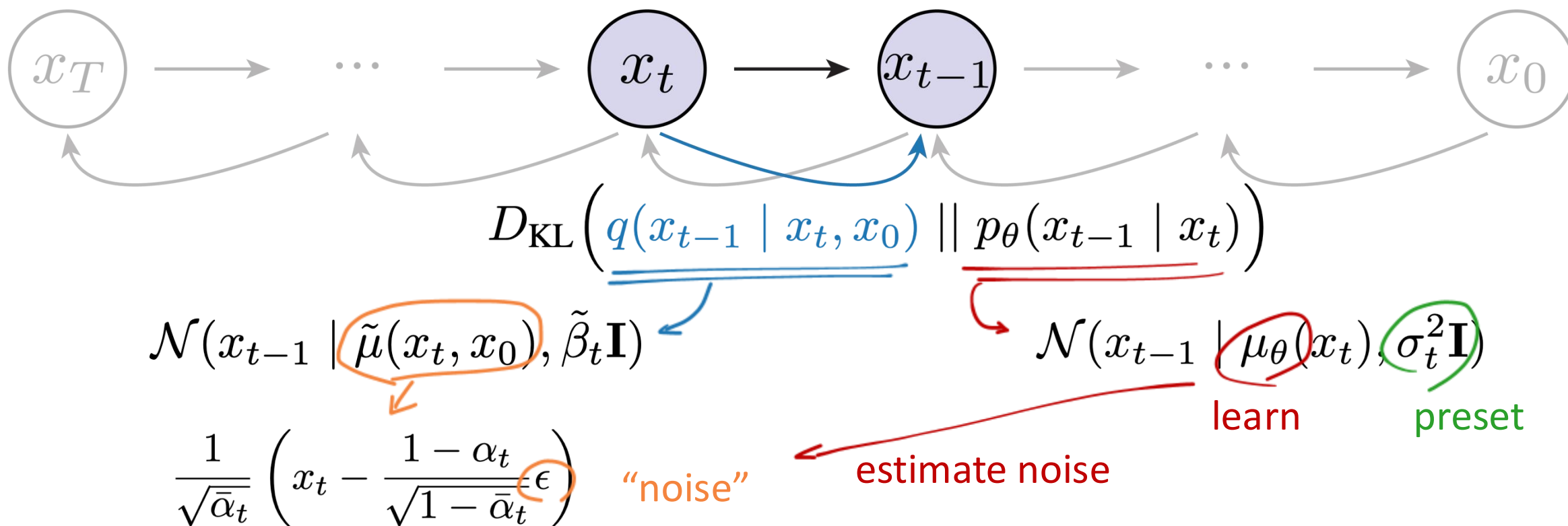
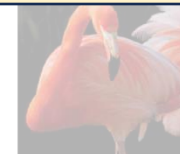
$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$



Reverse Process

D_{KL} of two Gaussians is like L2 loss: (pset 1)

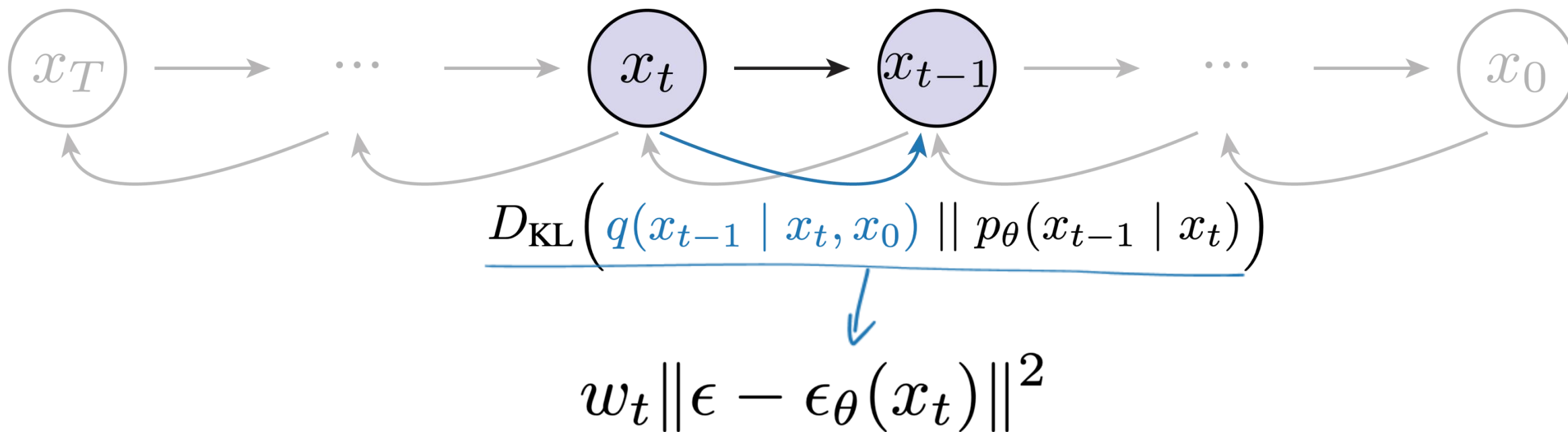
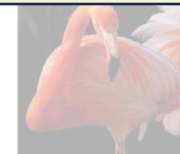
$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$



Reverse Process

D_{KL} of two Gaussians is like L2 loss: (pset 1)

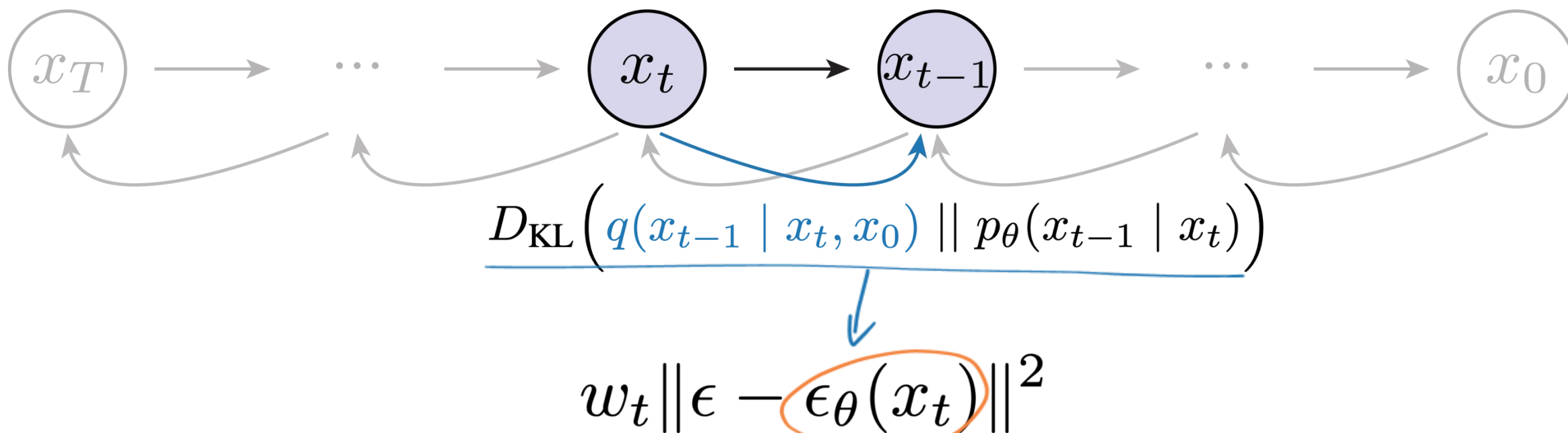
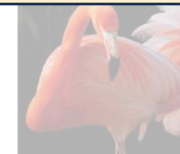
$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$



Reverse Process

D_{KL} of two Gaussians is like L2 loss: (pset 1)

$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$

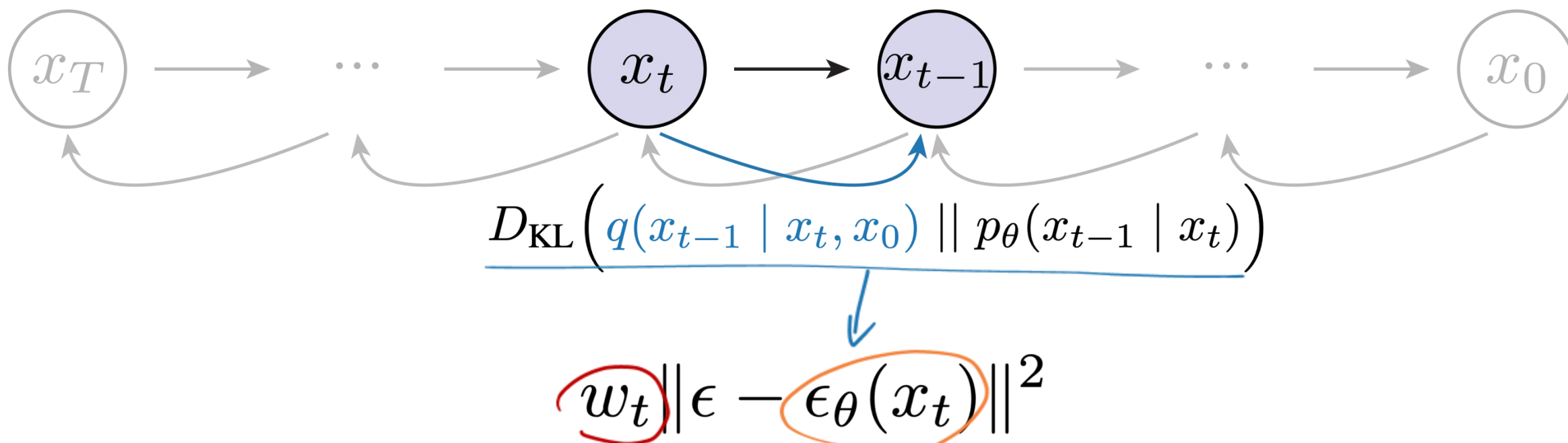
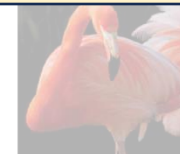


- a network to predict noise
- input: noisy image

Reverse Process

D_{KL} of two Gaussians is like L2 loss: (pset 1)

$$D_{KL}(\mathcal{N}_1 \parallel \mathcal{N}_2) = \log \left(\frac{\sigma_2}{\sigma_1} \right) + \frac{\sigma_1^2 + \|\mu_1 - \mu_2\|^2}{2\sigma_2^2} - \frac{1}{2}$$



- weights due to α_t, β_t
- but set as 1 (**critical**)
- a network to predict noise
- input: noisy image

tl; dr

- some dependency graphs
- some linear combinations
- D_{KL}
- L2 loss of noise

Diffusion Models

Forward process

- add noise to data

Reverse process

- learn to denoise

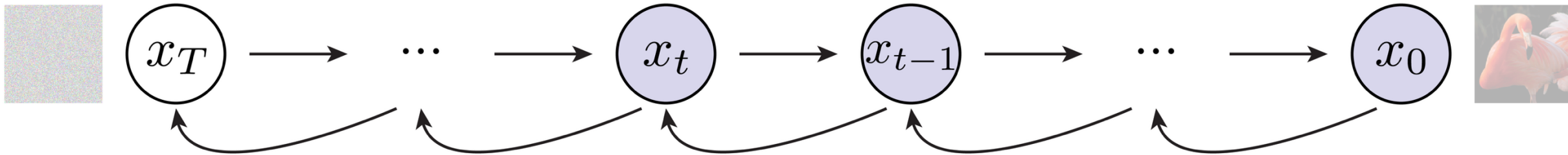
Training objective

- from Hierarchical VAE to L2 loss

Noise Conditional Network

- represent a distribution

Training Objective



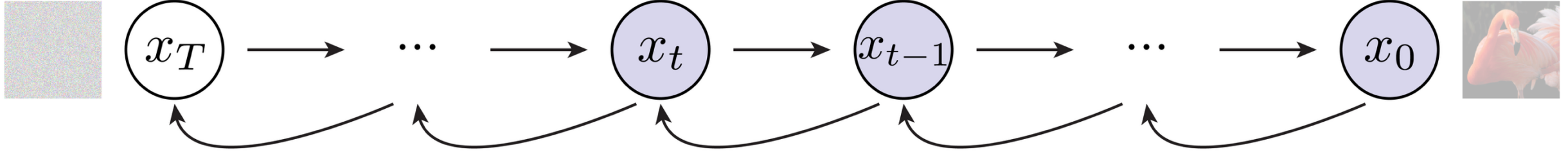
$$\mathcal{L}_{\text{VLB}} \quad := \quad \mathcal{L}_T + \mathcal{L}_{T-1} + \dots + \mathcal{L}_0$$

$$\mathcal{L}_T \quad := \quad D_{\text{KL}}\left(q(x_T \mid x_0) \parallel p_{\theta}(x_T)\right)$$

$$\mathcal{L}_{t-1} \quad := \quad D_{\text{KL}}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)\right)$$

$$\mathcal{L}_0 \quad := \quad -\log p_{\theta}(x_0 \mid x_1)$$

Training Objective



- variational lower bound
- like ELBO

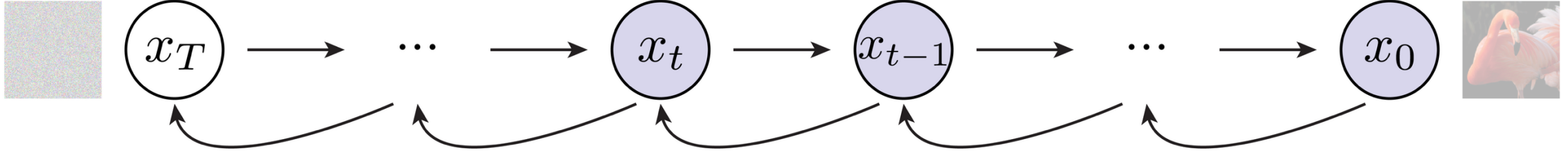
$$\mathcal{L}_{\text{VLB}} := \mathcal{L}_T + \mathcal{L}_{T-1} + \dots + \mathcal{L}_0$$

$$\mathcal{L}_T := D_{\text{KL}}\left(q(x_T \mid x_0) \parallel p_{\theta}(x_T)\right)$$

$$\mathcal{L}_{t-1} := D_{\text{KL}}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)\right)$$

$$\mathcal{L}_0 := -\log p_{\theta}(x_0 \mid x_1)$$

Training Objective



- variational lower bound
- like ELBO

$$\mathcal{L}_{\text{VLB}} := \mathcal{L}_T + \mathcal{L}_{T-1} + \dots + \mathcal{L}_0$$

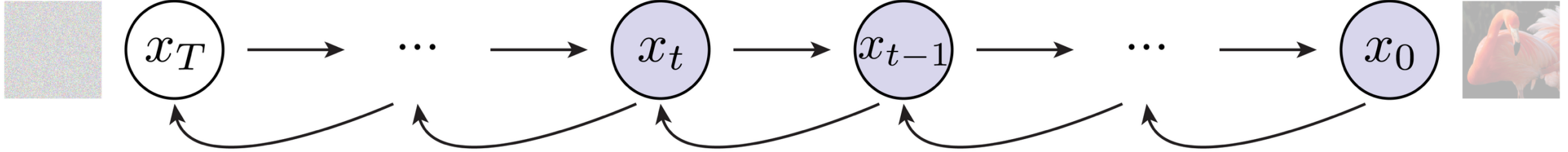
$$\mathcal{L}_T := D_{\text{KL}}\left(q(\overset{\text{Z}}{\cancel{x_T}} \mid x_0) \parallel p_{\theta}(\overset{\text{Z}}{\cancel{x_T}})\right)$$

if it's one-step,
this is ELBO.

~~$$\mathcal{L}_{t-1} := D_{\text{KL}}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)\right)$$~~

$$\mathcal{L}_0 := -\log p_{\theta}(x_0 \mid \overset{\text{Z}}{\cancel{x_1}})$$

Training Objective



$$\mathcal{L}_{\text{VLB}} := \mathcal{L}_T + \mathcal{L}_{T-1} + \dots + \mathcal{L}_0$$

no parameter, unlike VAE's q_ϕ

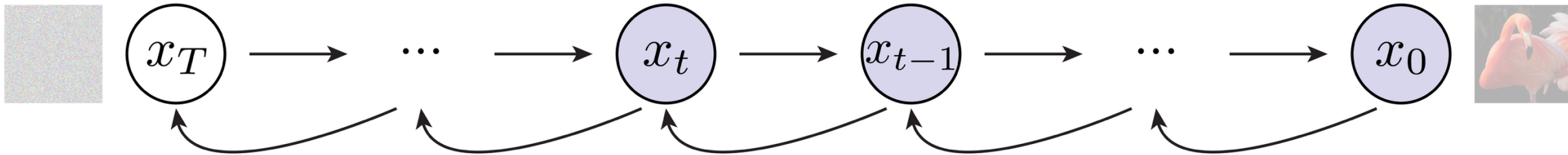
$$\text{constant } \mathcal{L}_T := D_{\text{KL}} \left(\underline{q(x_T | x_0)} \parallel \underline{p_\theta(x_T)} \right)$$

Gaussian

$$\mathcal{L}_{t-1} := D_{\text{KL}} \left(\underline{q(x_{t-1} | x_t, x_0)} \parallel p_\theta(x_{t-1} | x_t) \right)$$

$$\mathcal{L}_0 := -\log p_\theta(x_0 | x_1)$$

Training Objective



$$\mathcal{L}_{\text{VLB}} := \mathcal{L}_T + \mathcal{L}_{T-1} + \dots + \mathcal{L}_0$$

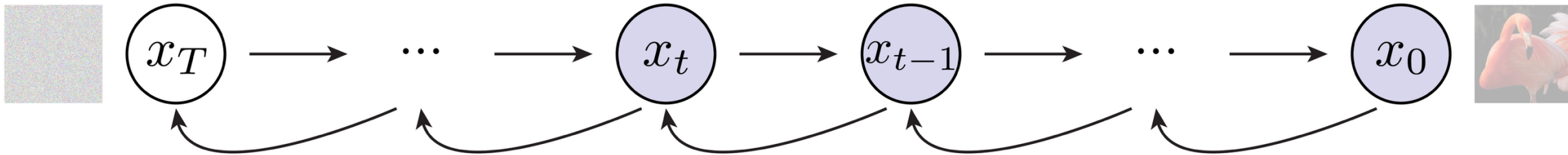
$$\mathcal{L}_T := D_{\text{KL}}\left(q(x_T \mid x_0) \parallel p_{\theta}(x_T)\right)$$

$$\mathcal{L}_{t-1} := D_{\text{KL}}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)\right)$$

reconstruction loss,
like VAE

$$\mathcal{L}_0 := -\log p_{\theta}(x_0 \mid x_1)$$

Training Objective



$$\mathcal{L}_{\text{VLB}} := \mathcal{L}_T + \mathcal{L}_{T-1} + \dots + \mathcal{L}_0$$

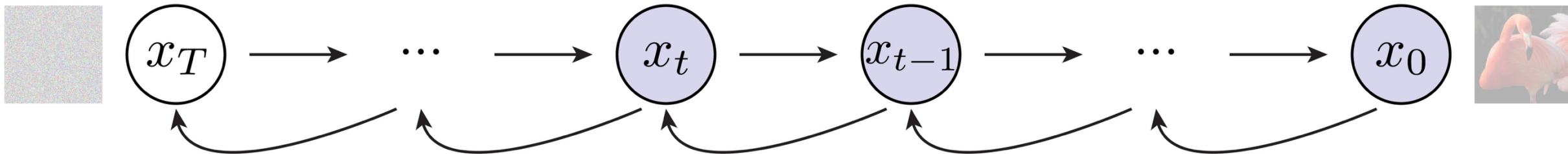
$$\mathcal{L}_T := D_{\text{KL}}\left(q(x_T \mid x_0) \parallel p_{\theta}(x_T)\right)$$

L2 loss on noise

$$\mathcal{L}_{t-1} := D_{\text{KL}}\left(q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t)\right)$$

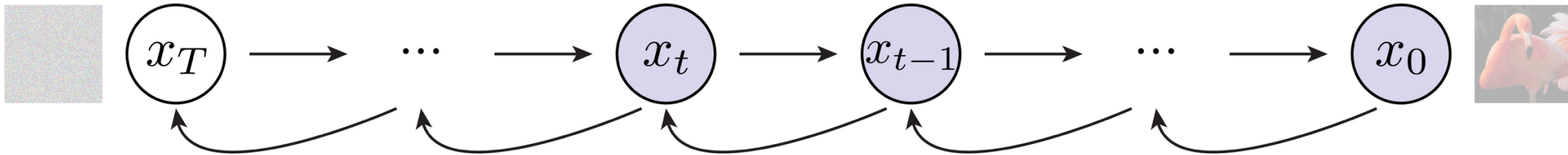
$$\mathcal{L}_0 := -\log p_{\theta}(x_0 \mid x_1)$$

Training Objective



$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \left[w_t \| \epsilon - \epsilon_{\theta}(x_t, t) \|^2 \right]$$

Training Objective

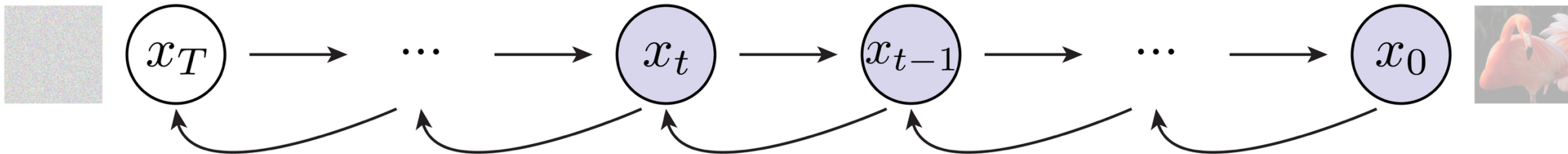


$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \left[w_t \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right]$$

Diagram illustrating the components of the expectation operator $\mathbb{E}_{x_0, t, \epsilon}$:

- Red arrow: over p_{data} (points to x_0)
- Green arrow: over $[1, T]$ (points to t)
- Orange arrow: over $\mathcal{N}(0, \mathbf{I})$ (points to ϵ)

Training Objective



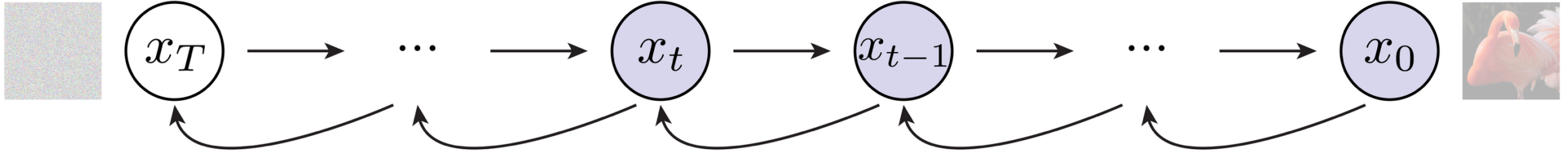
$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \left[\underbrace{w_t}_{\text{set as 1 (critical)}} \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right]$$

set as 1 (critical)

Objective	IS	FID
L , learned diagonal Σ	–	–
L , fixed isotropic Σ	7.67 ± 0.13	13.51
$\ \tilde{\epsilon} - \epsilon_\theta\ ^2$ (L_{simple})	9.46 ± 0.11	3.17

[Ho et al. 2020]; see more in [Salimans & Ho, 2022]

Training Objective



$$\mathcal{L} = \mathbb{E}_{x_0, t, \epsilon} \left[w_t \left\| \epsilon - \epsilon_{\theta}(x_t, t) \right\|^2 \right]$$

network to
predict noise

conditioned on
noise level (**critical**)

Diffusion Models

Forward process

- add noise to data

Reverse process

- learn to denoise

Training objective

- from Hierarchical VAE to L2 loss

Noise Conditional Network

- represent a distribution

Noise Conditional Network

- Diffusion models decompose a distribution into **many** simpler ones.
- We need the same number of networks to fit **all** of them.
- We can **combine** all into **one** network.
- This network is conditioned on noise level t .
- **Noise Conditional Network** [Song & Ermon 2019]: things made work

*It is called Noise Conditional Score Network (NCSN) in [Song & Ermon 2019] in the context of score matching.

Noise Conditional Network

How to represent $p_{\theta}(x_{t-1} \mid x_t)$

- network input: x_t
- network output: μ and σ of a distribution
- parametrize μ by: $\epsilon_{\theta}(x_t, t)$

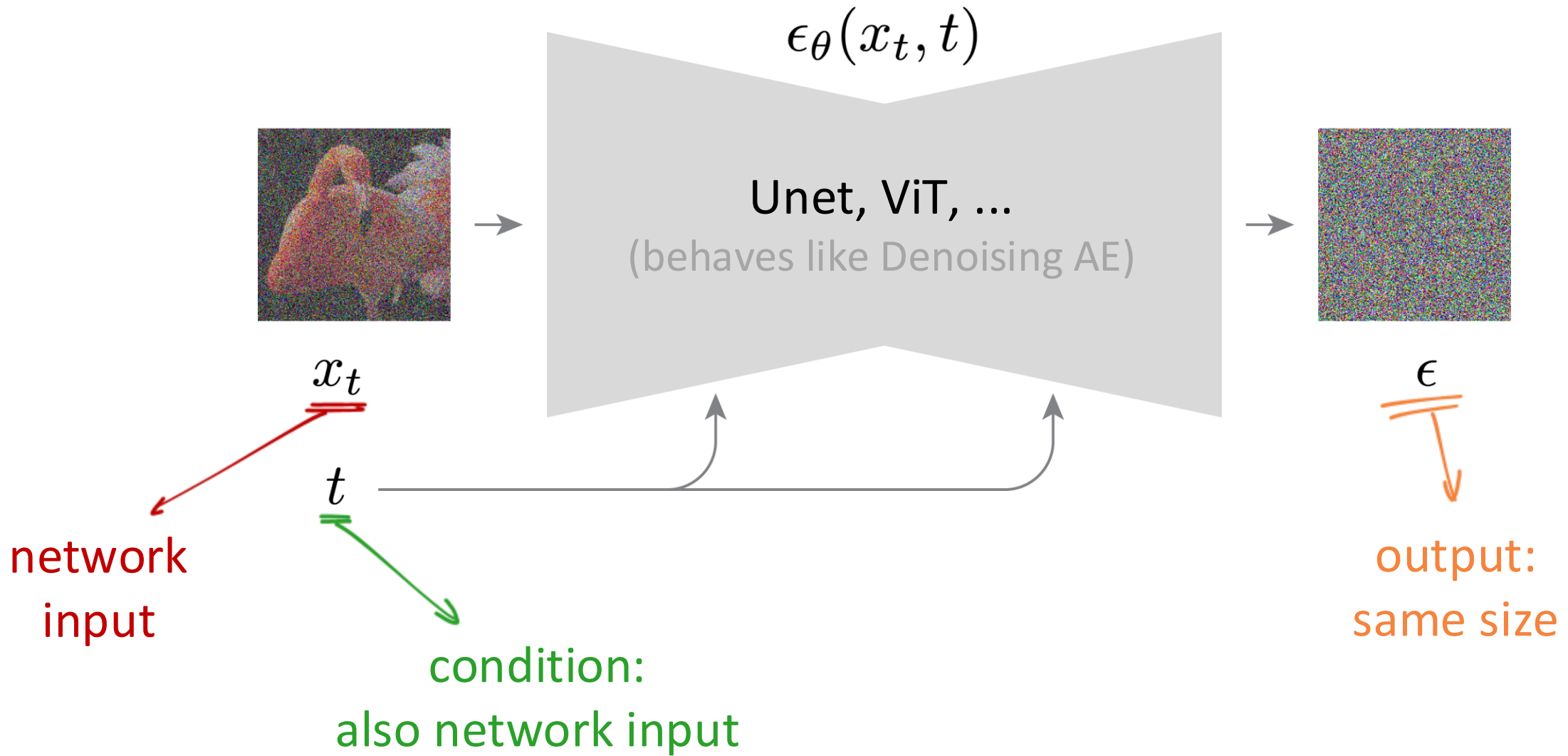
noisy image:

- condition
- network input

noise level:

- condition
- network input

Noise Conditional Network



Diffusion Models: Put Everything Together

Forward process

- add noise to data

Reverse process

- learn to denoise

Training objective

- from Hierarchical VAE to L2 loss

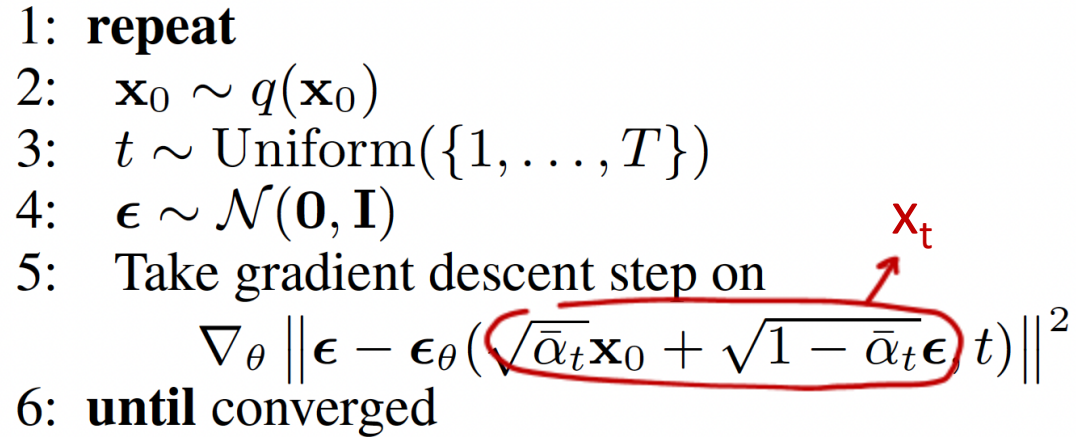
Noise Conditional Network

- represent a distribution

Diffusion algorithm annotated:

Algorithm 1 Training

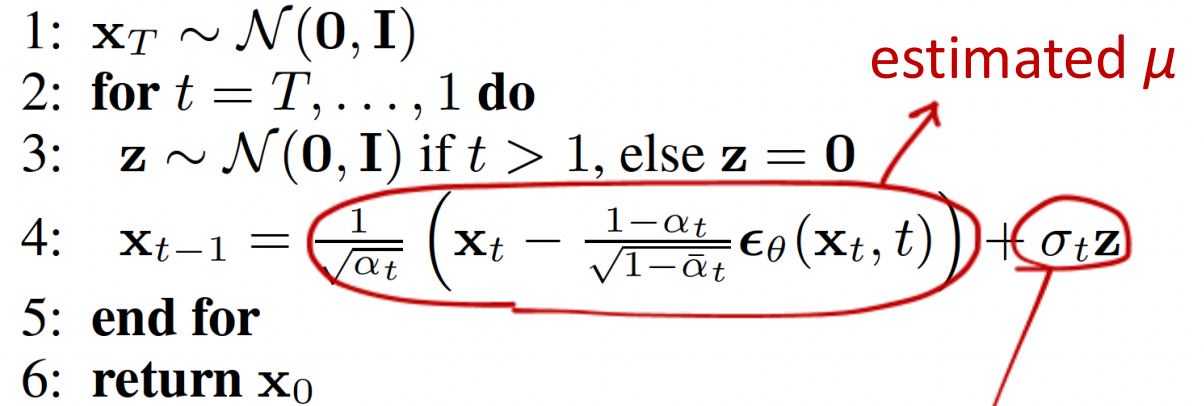
```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
6: until converged
```



The training algorithm is annotated with red circles and arrows. A red circle highlights the expression $\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ in the loss function, with a red arrow pointing to $\mathbf{x}_t$. Another red circle highlights the noise vector ϵ in the sampling step, with a red arrow pointing to it.

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```



The sampling algorithm is annotated with red circles and arrows. A red circle highlights the expression $\frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$, with a red arrow pointing to the text "estimated μ ". Another red circle highlights the term $\sigma_t \mathbf{z}$, with a red arrow pointing to the text "sampling from estimated distribution".

sampling from
estimated distribution

Diffusion algorithm annotated:

Algorithm 1 Training

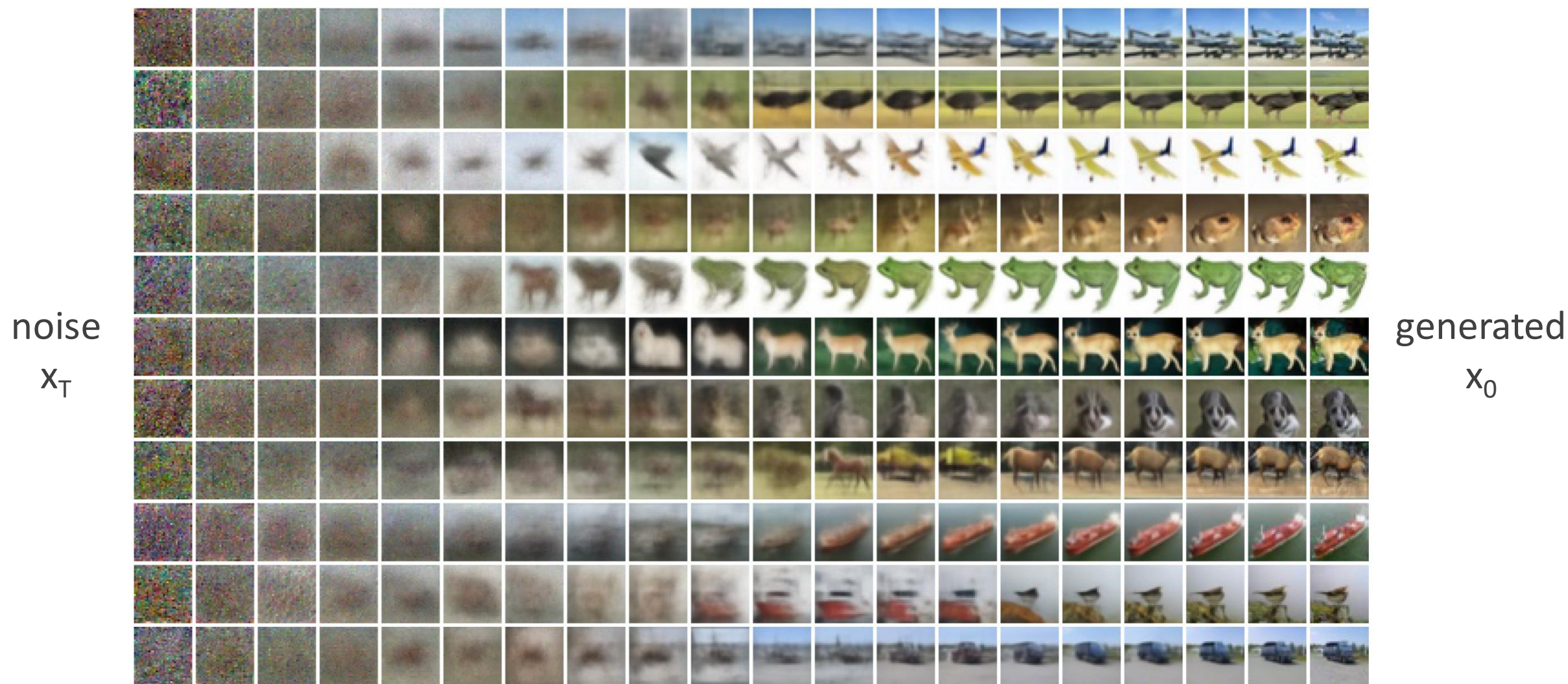
```
1: repeat  
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$   
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$   
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
5:   Take gradient descent step on  
        $\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$   
6: until converged
```

Algorithm 2 Sampling

```
1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$   
2: for  $t = T, \dots, 1$  do  
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$   
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$   
5: end for  
6: return  $\mathbf{x}_0$ 
```

tl; dr: just noising and denoising

Example: Unconditional Generation on CIFAR-10



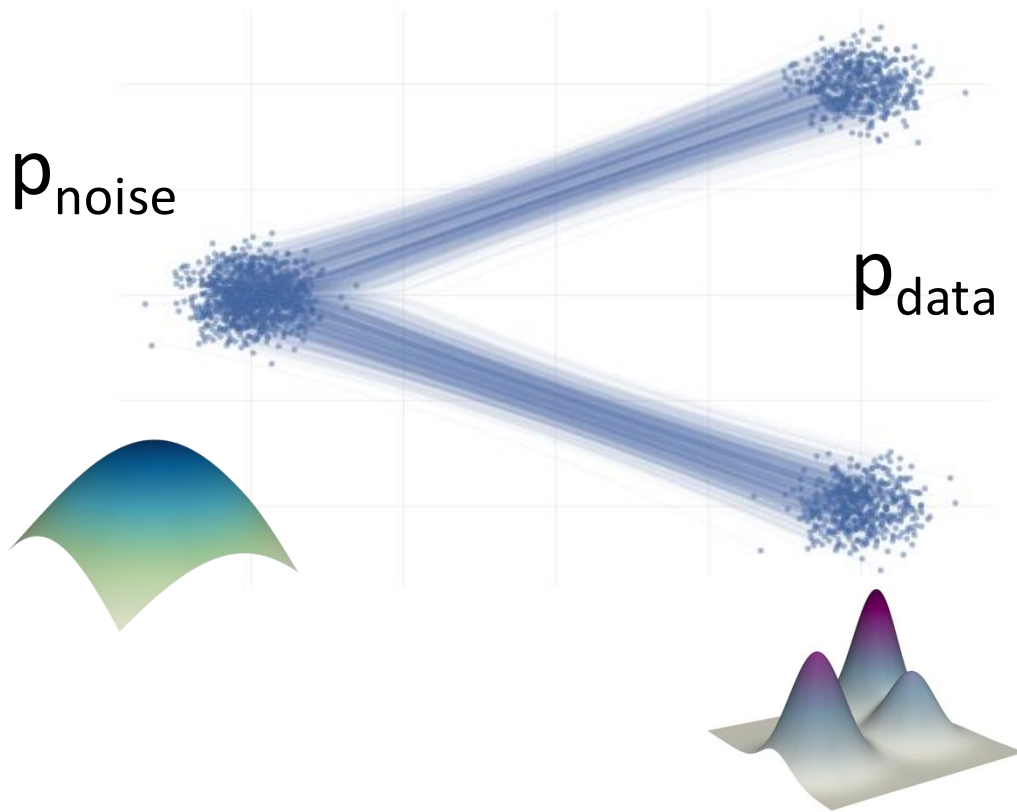
Flow Matching

Flow Matching: Introduction

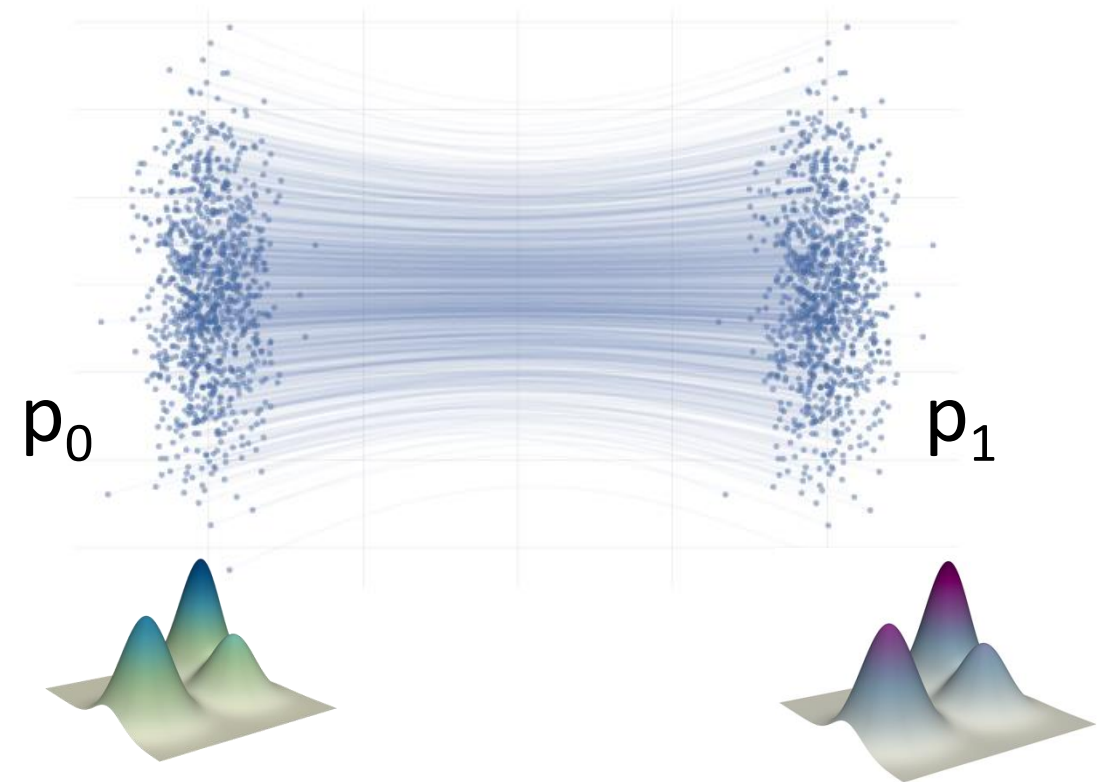
- Originate from **Continuous Normalizing Flows** (CNF)
- Closely related to Diffusion
 - Diffusion: **noise** prediction
 - Flow Matching: **velocity** prediction
- **Simpler** objective and implementation
- Often used **interchangeably** with Diffusion

Flowing between Distributions

- Noise to Data



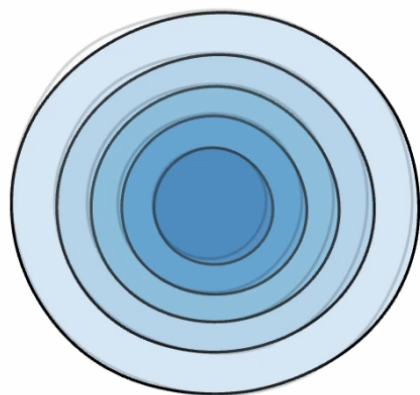
- Conceptually:
Any to Any



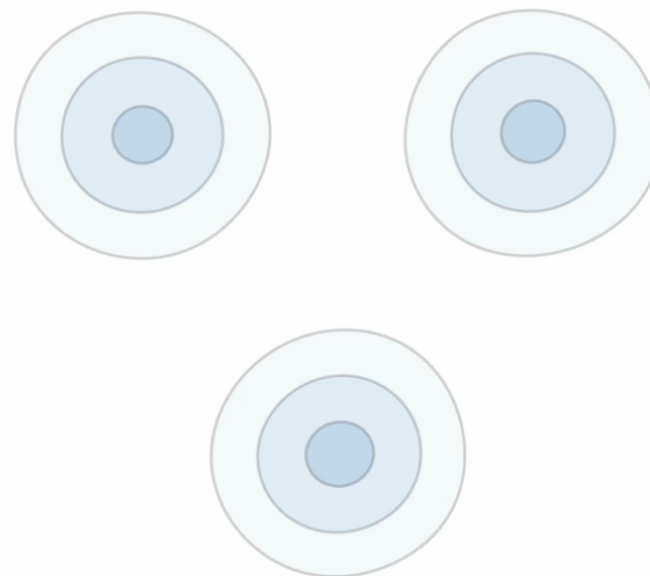
Flow Matching

The Evolution of the Marginal Probability Path

Source Distribution



Target Distribution



0

Time

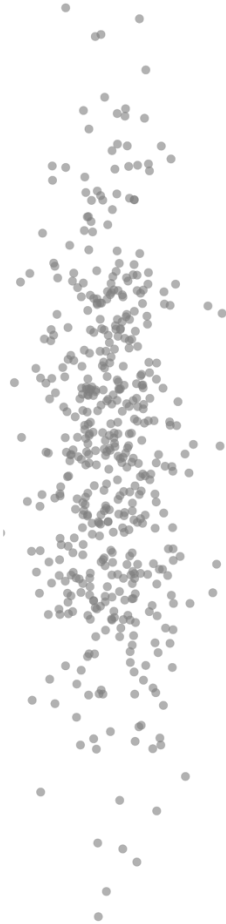
1

Video source: Diffusion Explorer: Interactive Visualizations of Diffusion and Flow Based Generative Models

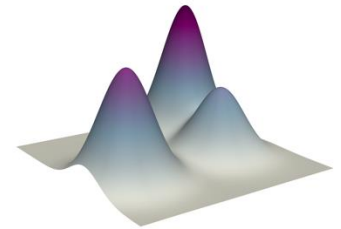
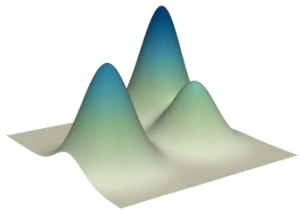
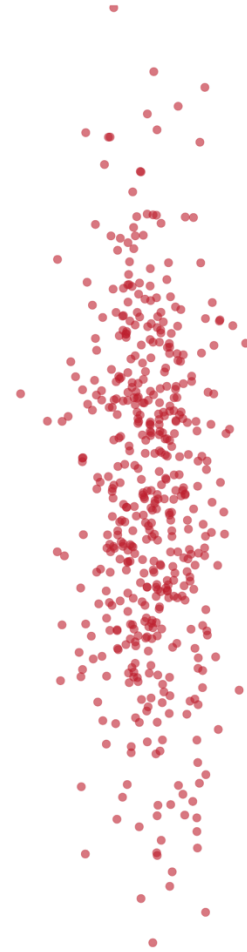
<https://github.com/helblazer811/Diffusion-Explorer>

Flowing between Distributions

data



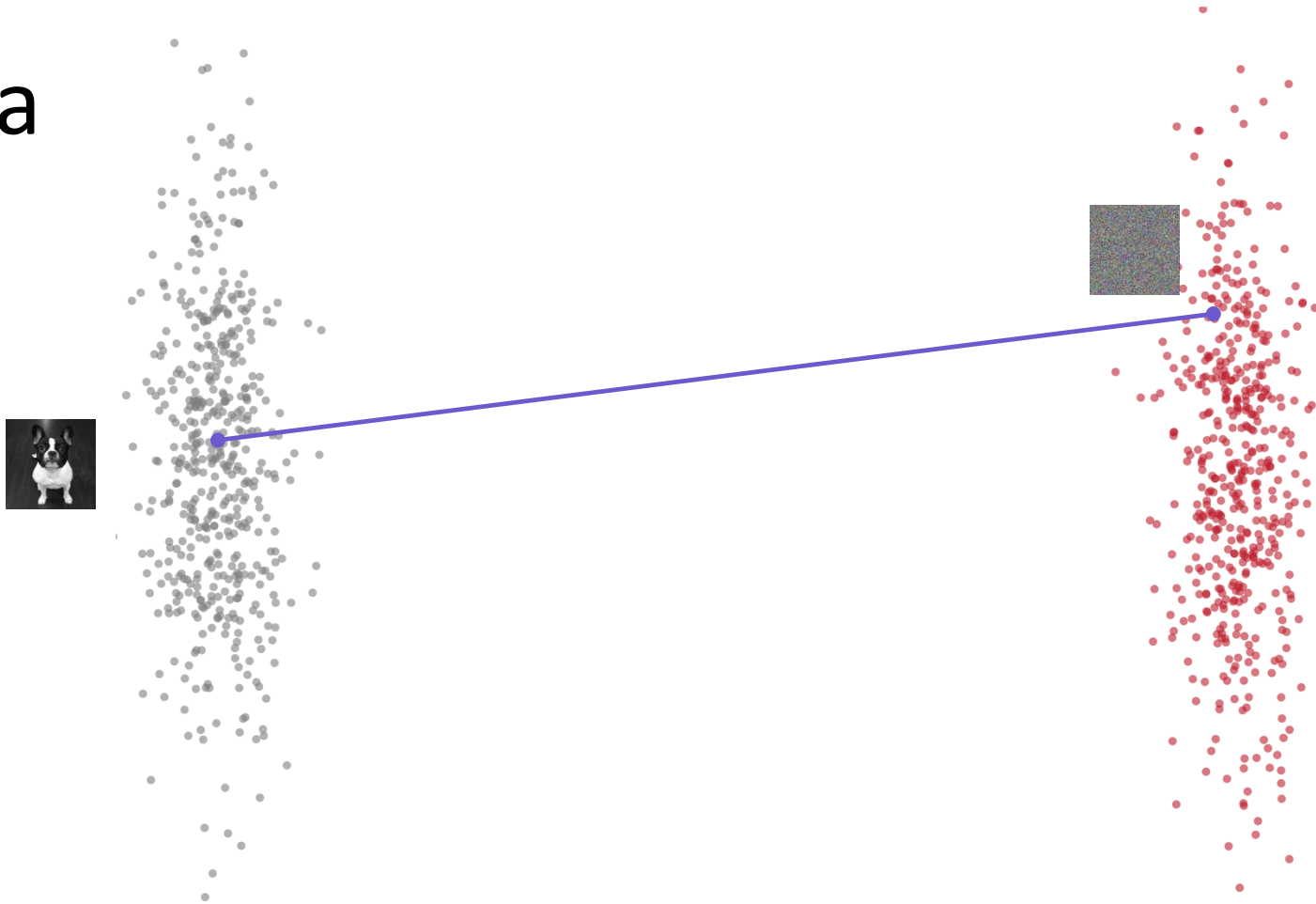
noise
(prior)



Flowing between Distributions

data

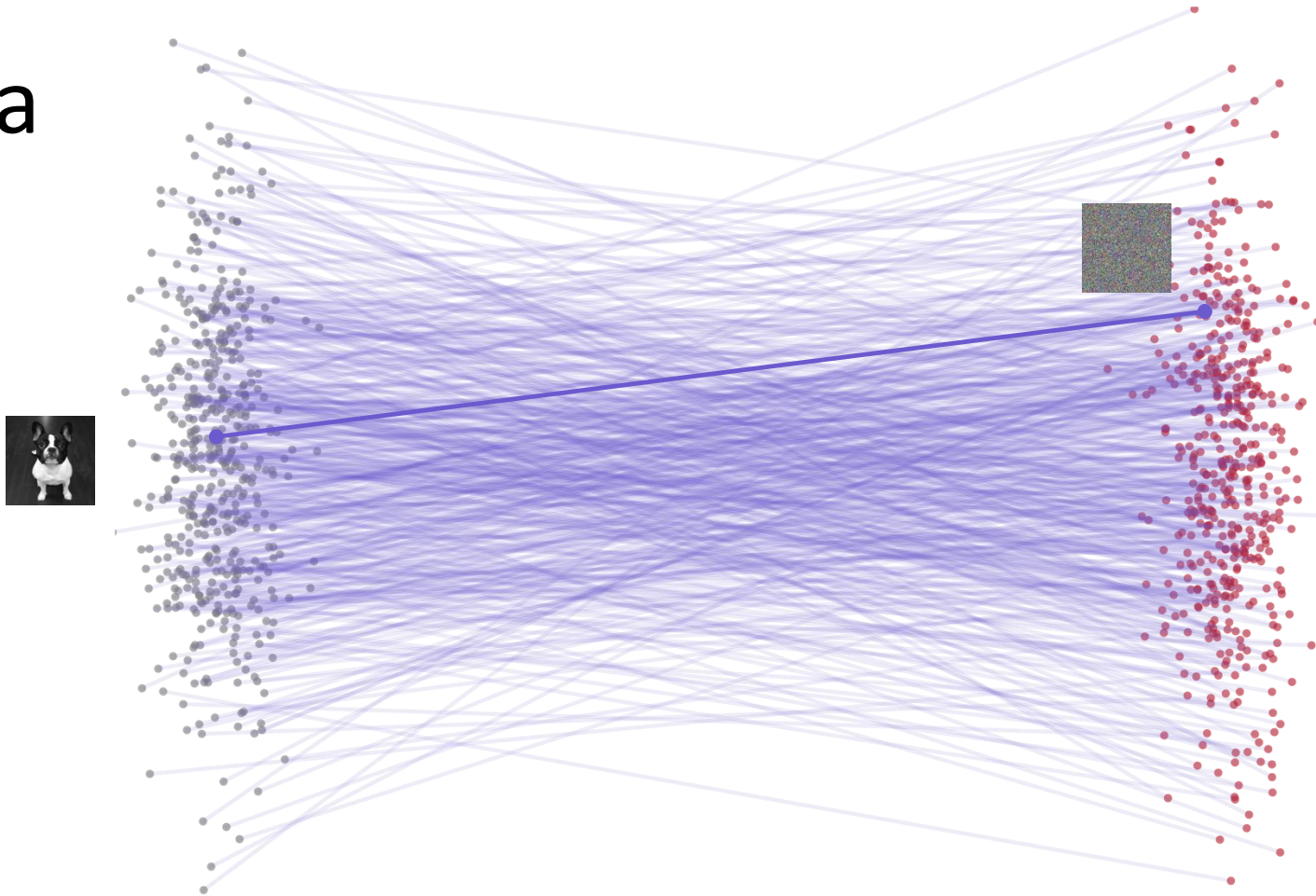
noise



Flowing between Distributions

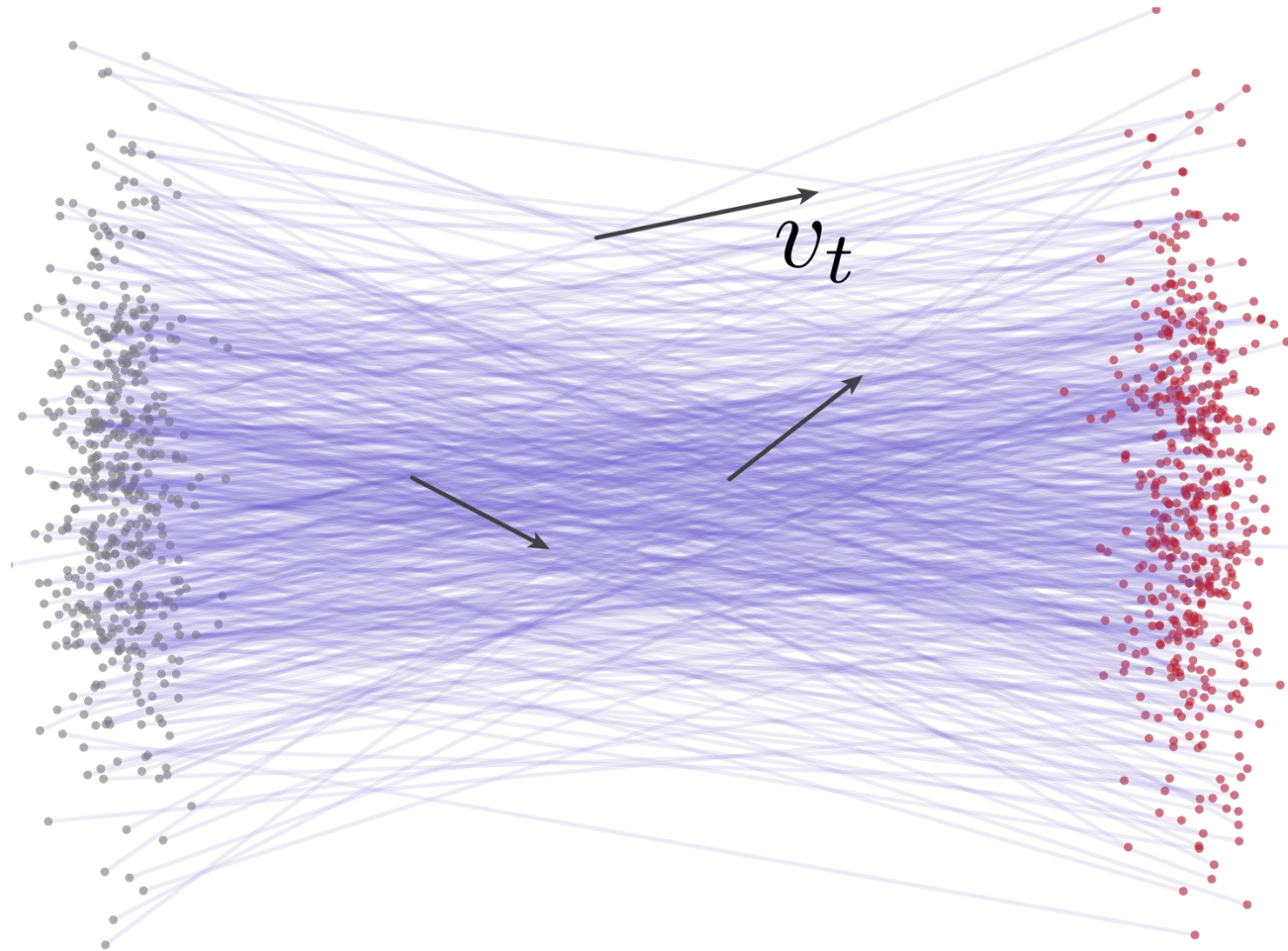
data

noise

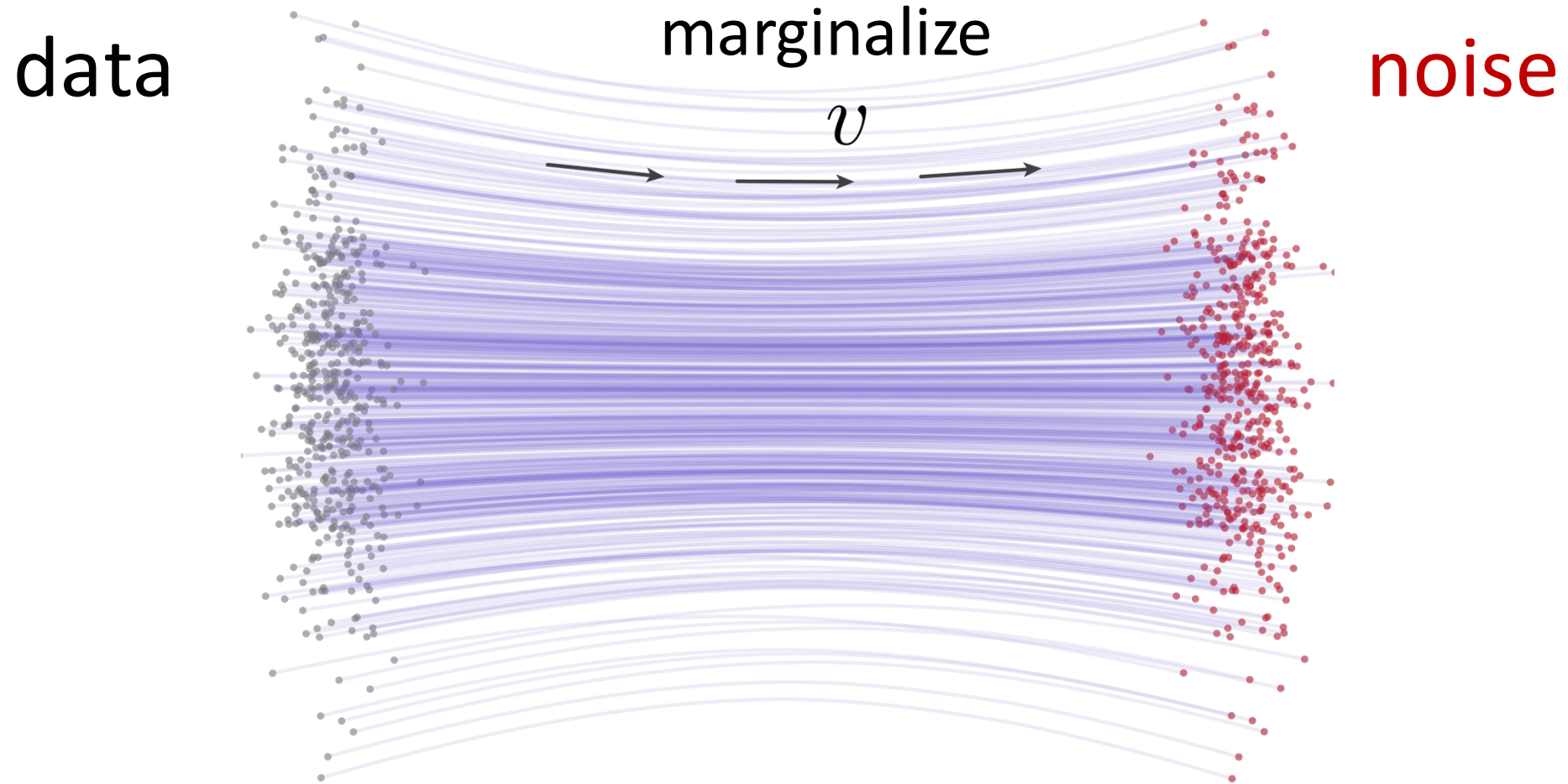


Flowing between Distributions

data

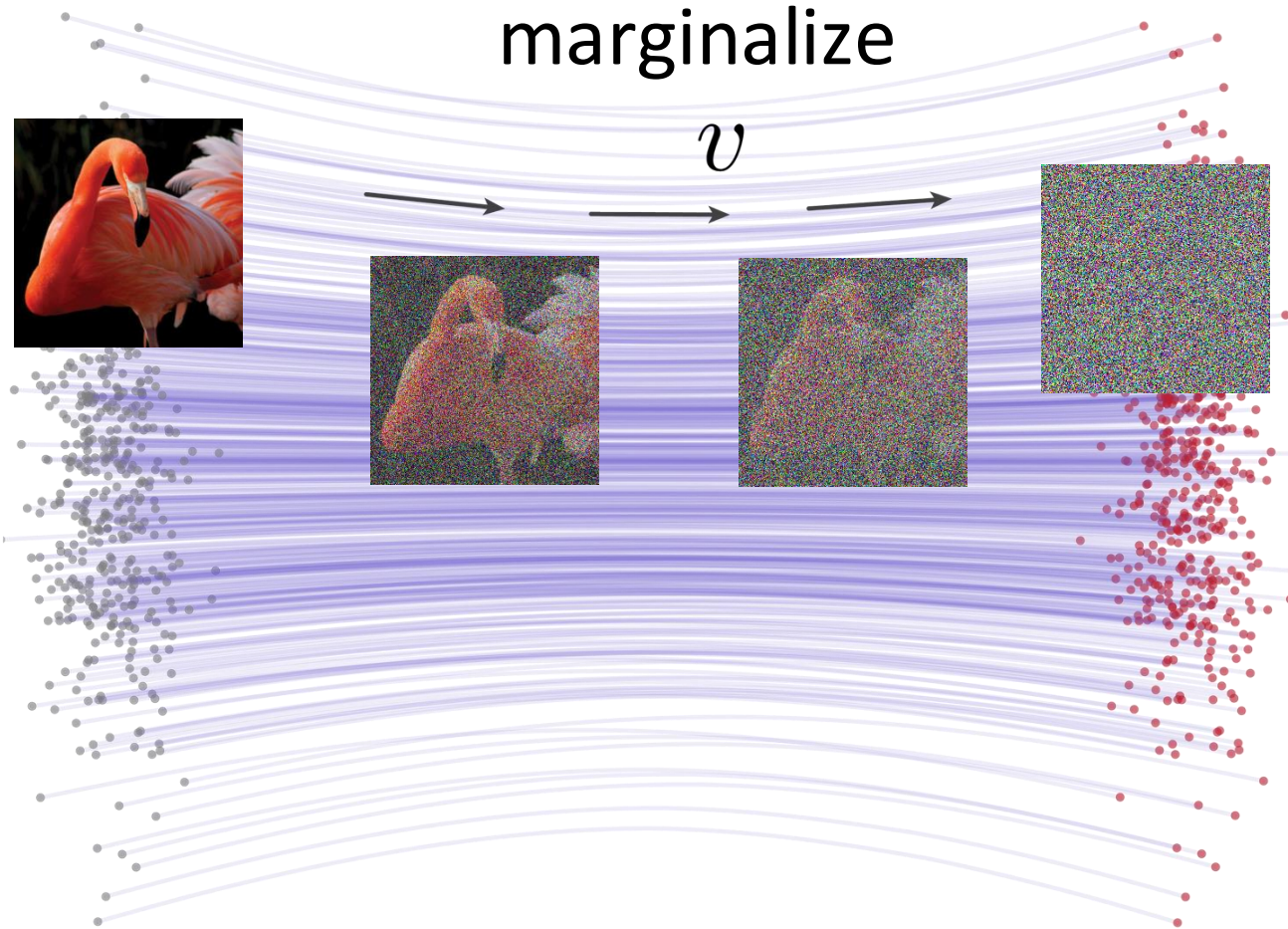


Flowing between Distributions

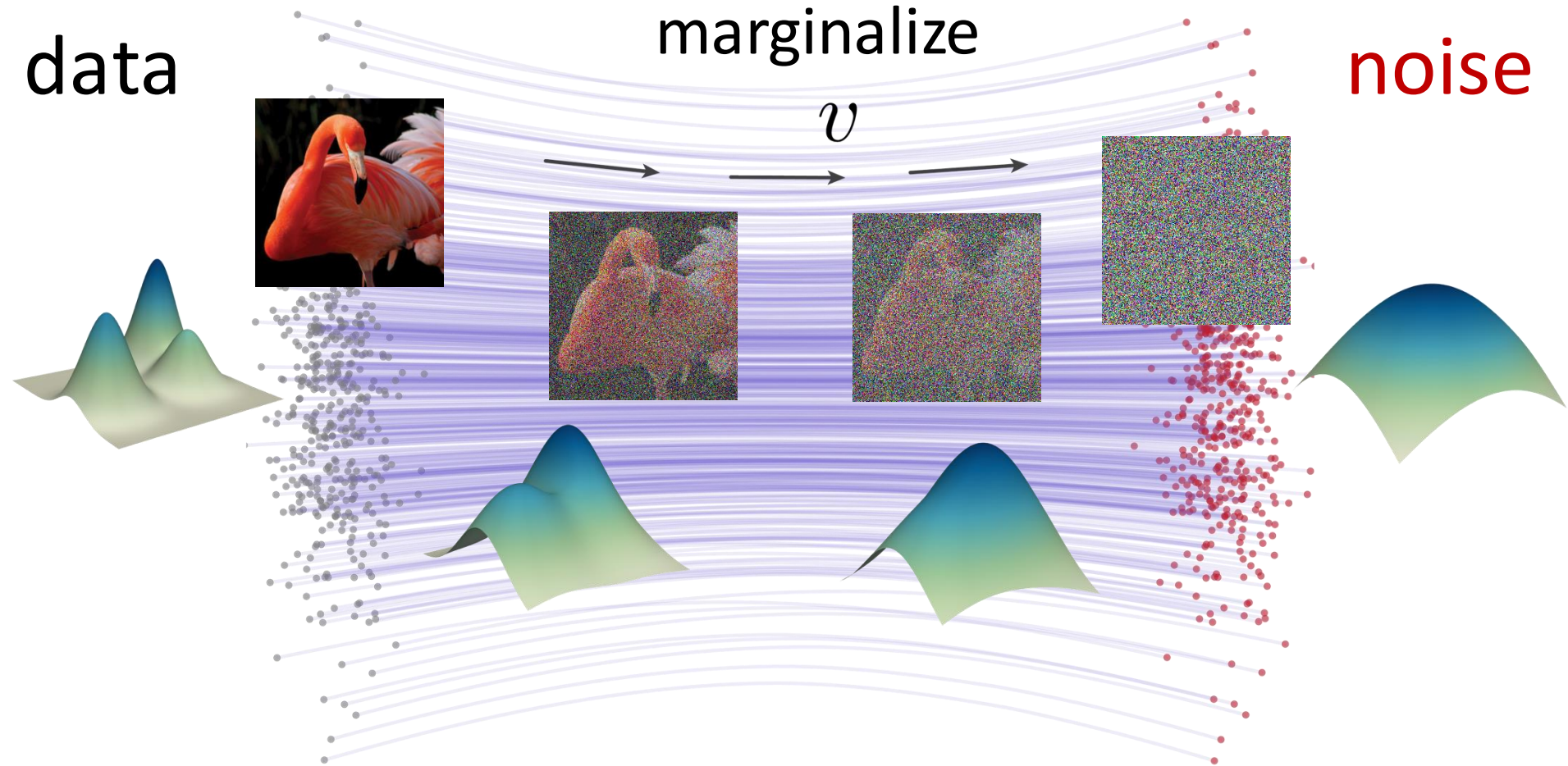


Flowing between Distributions

data



Flowing between Distributions



Flow Matching

data



x

interpolation:

$$z_t = a_t x + b_t \epsilon$$

ϵ



noise

a_t, b_t : schedules

Example: **linear** schedule

$$z_t = (1 - t) x + t \epsilon$$

$$t \in [0, 1]$$

Flow Matching

data



x

interpolation:

$$z_t = a_t x + b_t \epsilon$$

ϵ



noise

velocity:

$$v_t = z'_t = a'_t x + b'_t \epsilon$$

Example: **linear** schedule

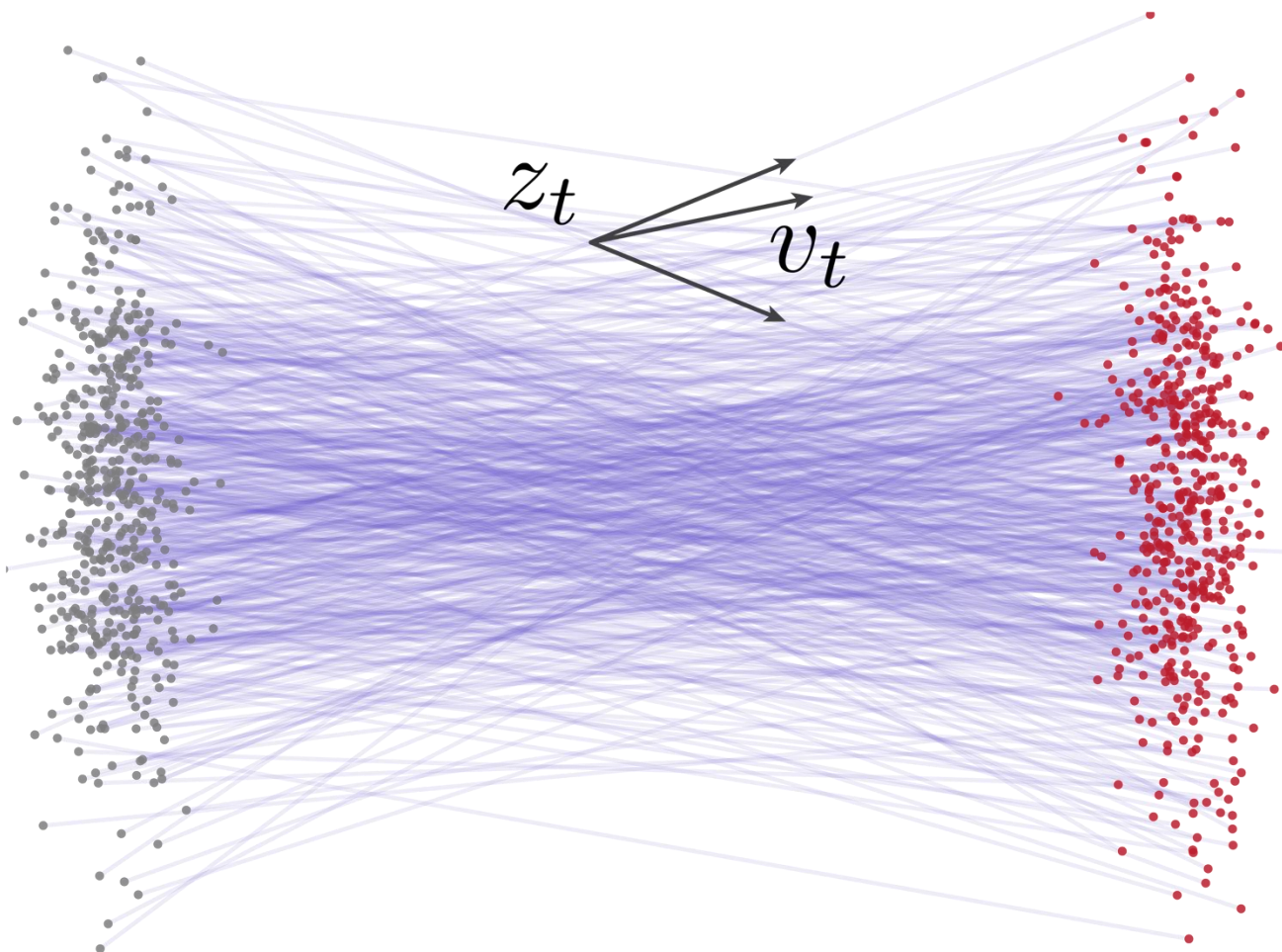
$$z_t = (1 - t) x + t \epsilon$$

$$v_t = \epsilon - x$$

Flow Matching

data

noise

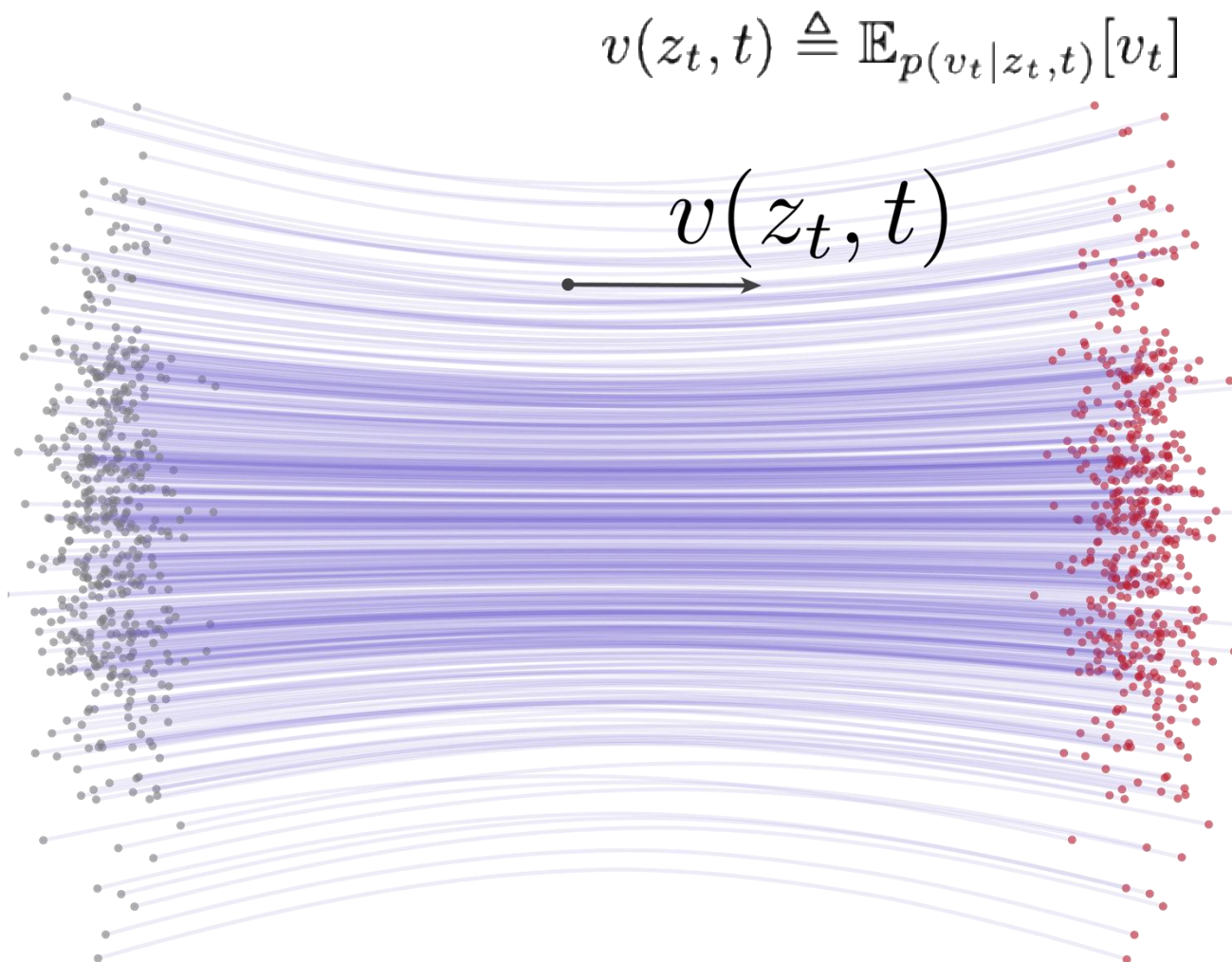


Conditional
Flow Matching:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E} \|v_{\theta}(z_t, t) - v_t(z_t|x)\|^2$$

Flow Matching

data

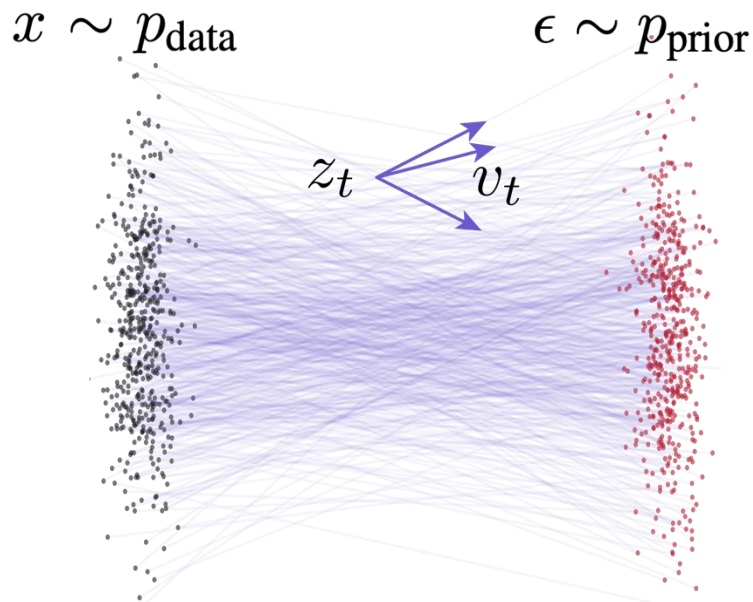


(Marginal)
Flow Matching:

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \|v_{\theta}(z_t, t) - v(z_t, t)\|^2$$

$\mathcal{L}_{\text{FM}}$
has the same
gradients as
 $\mathcal{L}_{\text{CFM}}$

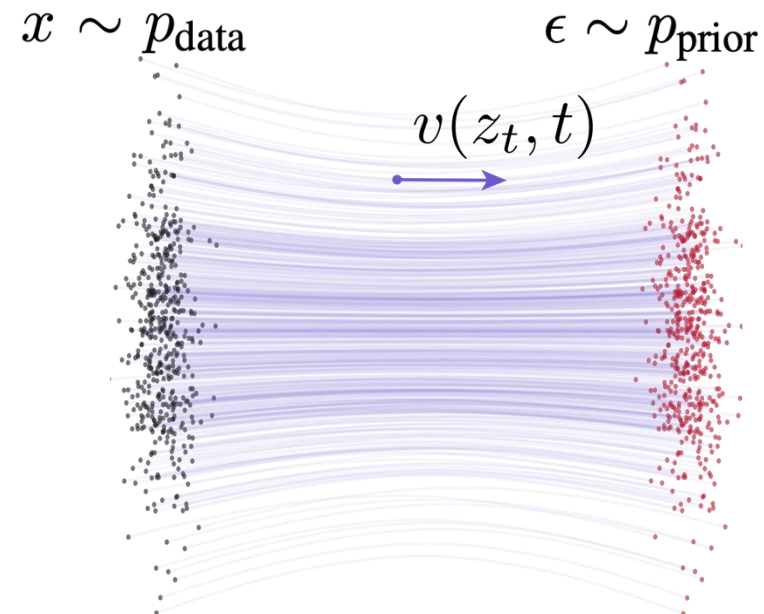
Flow Matching



$$z_t = (1 - t)x + t\epsilon$$

conditional velocity: $v_t = \epsilon - x$

$$\mathcal{L}_{\text{CFM}} = \mathbb{E} \|v_{\theta}(z_t, t) - v_t\|^2$$



marginal velocity: $v(z_t, t) \triangleq \mathbb{E}_{p(v_t|z_t, t)}[v_t]$

$$\mathcal{L}_{\text{FM}} = \mathbb{E} \|v_{\theta}(z_t, t) - v(z_t, t)\|^2$$



Illustration inspired by: Fjelde, Mathieu, Dutordoir, “An Introduction to Flow Matching”

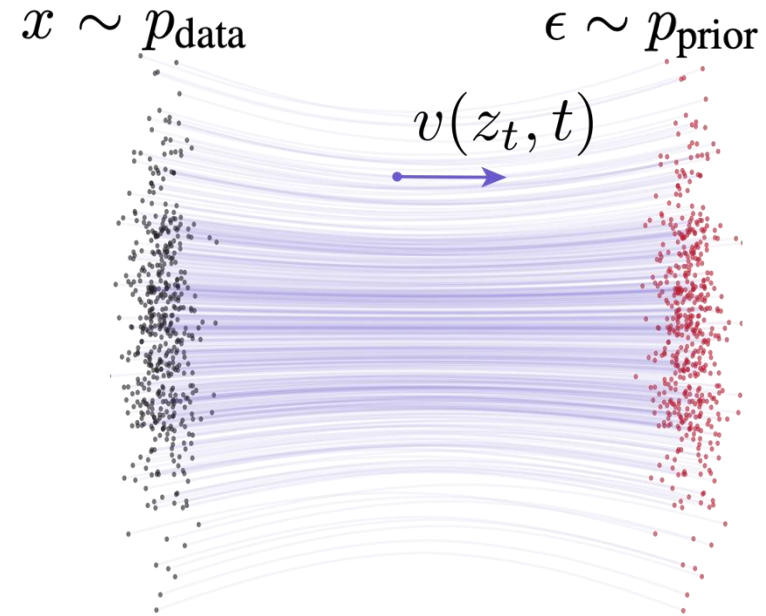
<https://mlg.eng.cam.ac.uk/blog/2024/01/20/flow-matching.html>

Flow Matching: Generation

Solve ODE for z_t

$$\frac{d}{dt}z_t = v(z_t, t)$$

- In principle, w/ **ground-truth** field $v(z_t, t)$
- In practice, approximate by $v_\theta(z_t, t)$



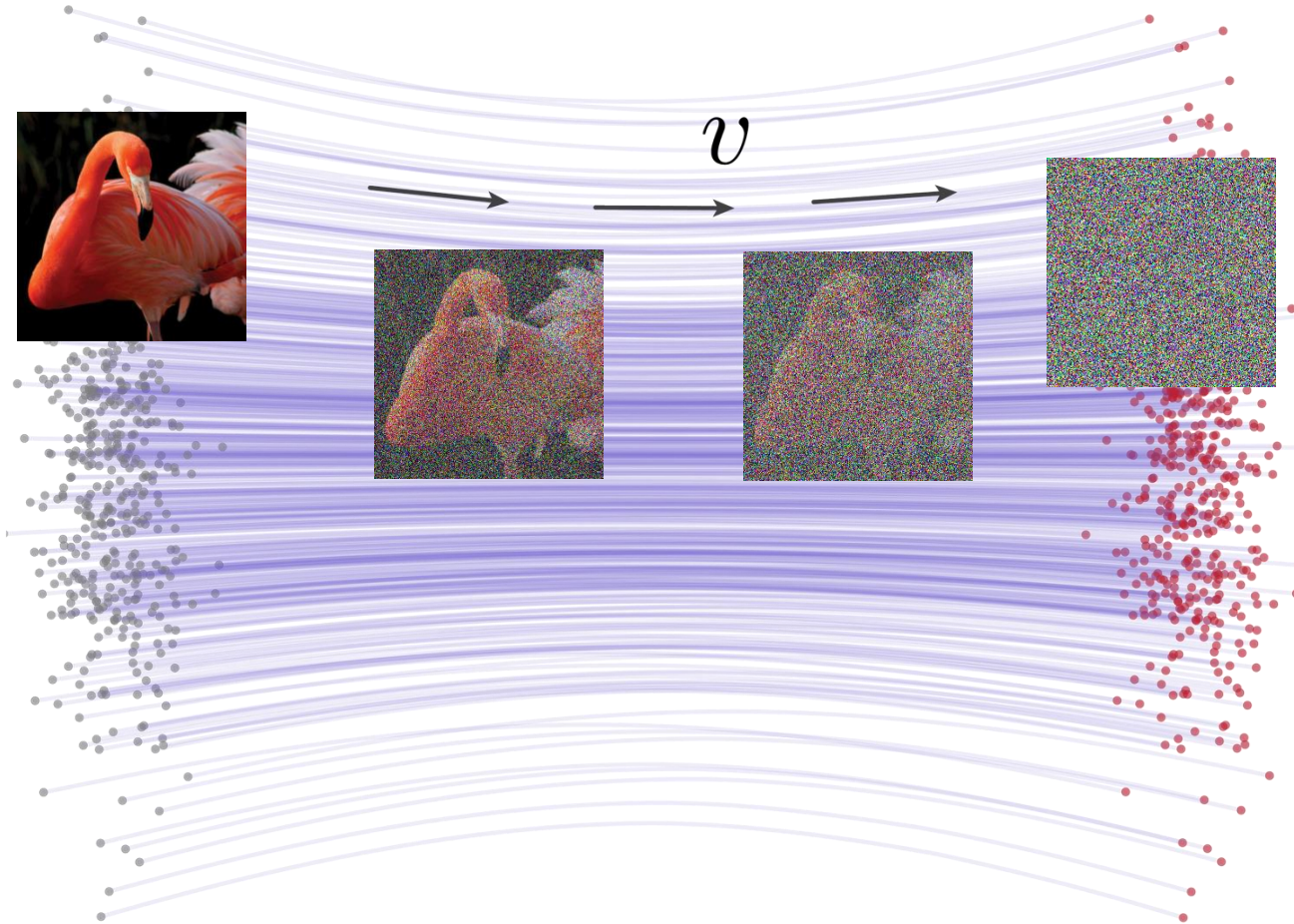
- Ideally, trajectory given by **integral**: $z_r = z_t - \int_r^t v(z_\tau, \tau) d\tau$

- In practice, approximate by numerical solver (e.g., Euler or Heun):

$$z_r = z_t + (r - t)v(z_t, t)$$

Flow Matching: Generation

data



Relation: Flow Matching and Diffusion

Flow Matching:

$$z_t = a_t x + b_t \epsilon$$

$$v_t = a'_t x + b'_t \epsilon$$

$$\mathcal{L} = \mathbb{E} \left[\|v_\theta(z_t, t) - v_t\|^2 \right]$$

Solve ODE: $\frac{d}{dt} z_t = v_\theta(z_t, t)$

Viewed as Diffusion:

$$z_t = a_t x + b_t \epsilon$$

reparametrize

$$\overset{\epsilon_\theta}{\cancel{\epsilon}} = \frac{a_t}{a_t b'_t - a'_t b_t} \left(\overset{v_\theta}{\cancel{v_t}} - \frac{a'_t}{a_t} z_t \right)$$

$$\mathcal{L} = \mathbb{E} \left[w_t^2 \|\epsilon_\theta(z_t, t) - \epsilon\|^2 \right]$$

$$v_\theta = \frac{a'_t}{a_t} z_t + \frac{a_t b'_t - a'_t b_t}{a_t} \epsilon_\theta$$

same



rewrite ϵ in z, v



in ϵ space

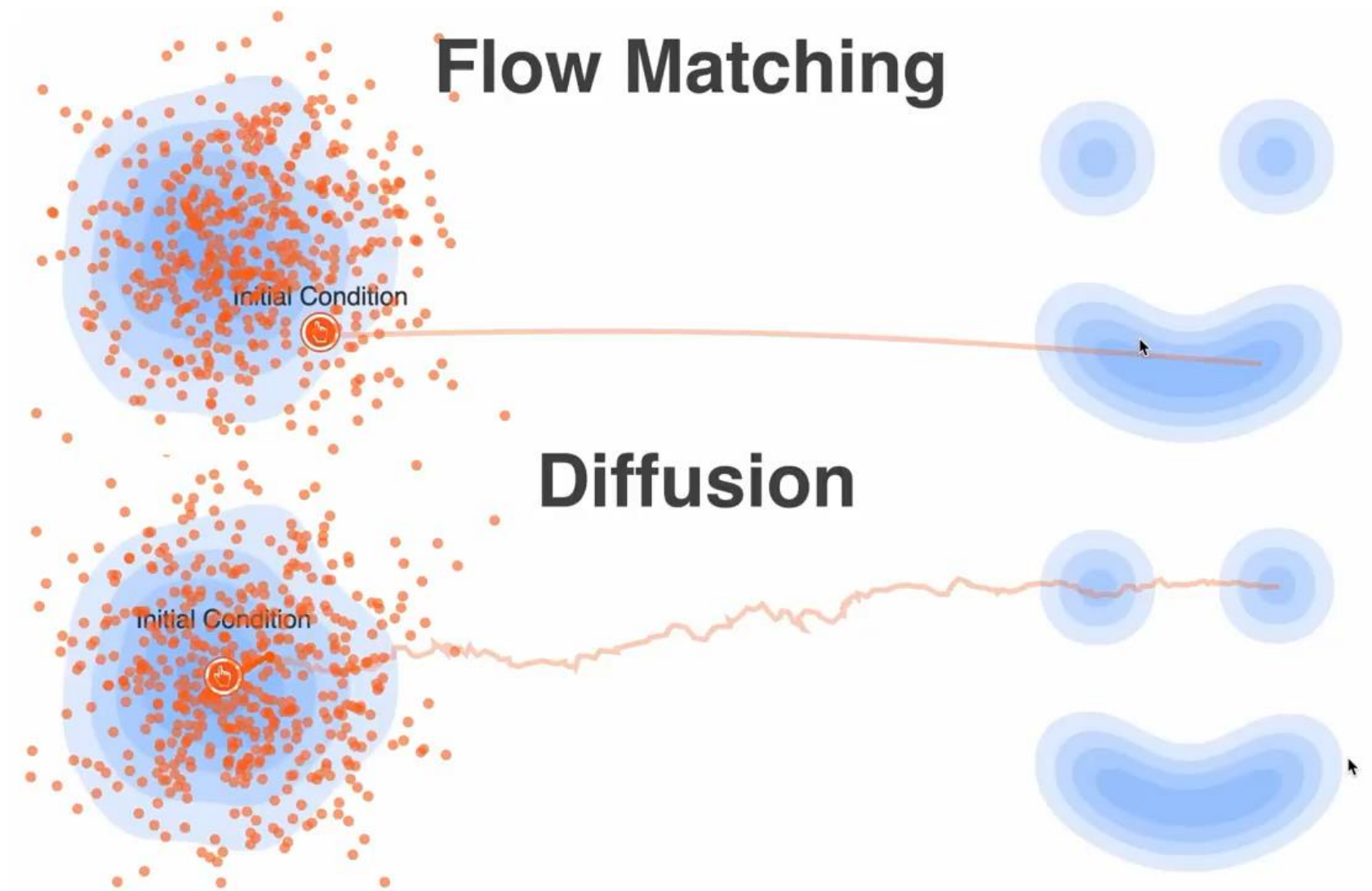


solve ODE in v space

Relation: Flow Matching and Diffusion

- **Reparameterization:** v-pred vs. e-pred
- **Loss reweighting:** v-loss vs. e-loss
- **Linear schedule:** simpler and good (applicable to both)
- **Continuous time + t-derivative:** easier to derive (applicable to both)

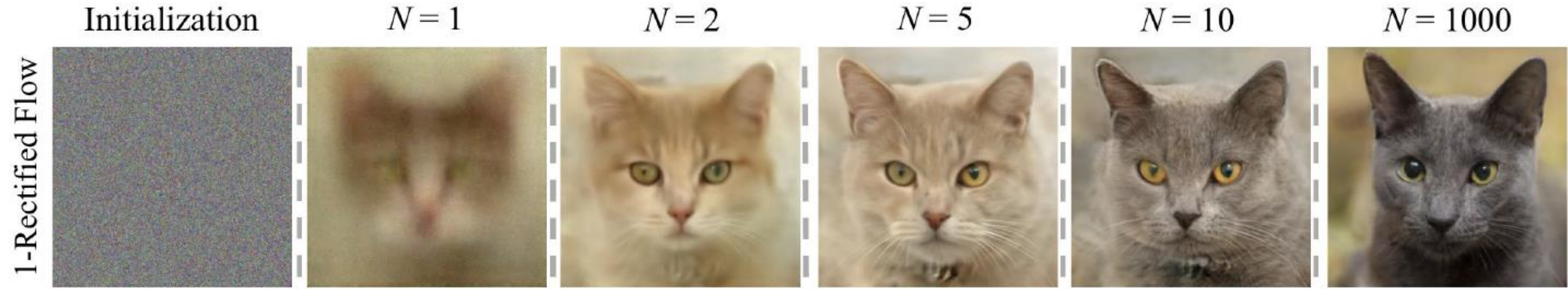
Today, both are often used interchangeably.



*Difference here is also caused by ODE vs. SDE solvers

Source: Alec Helbling @ X

Generation: Noise to Data



Flow between two distributions



Various Perspectives on Diffusion Models ...

- Hierarchical VAE
- Energy-based Models and Score Matching
- Autoregressive models

be a fully expressive conditional distribution. With these choices, $D_{\text{KL}}(q(\mathbf{x}_T) \parallel p(\mathbf{x}_T)) = 0$, and minimizing $D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t) \parallel p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t))$ trains p_{θ} to copy coordinates $t + 1, \dots, T$ unchanged and to predict the t^{th} coordinate given $t + 1, \dots, T$. Thus, training p_{θ} with this particular diffusion is training an autoregressive model.

[Ho et al, 2020]

- SDE and ODE
- Normalizing Flows
- Recurrent Neural Networks

See: Dieleman, “Perspectives on diffusion”. <https://sander.ai/2023/07/20/perspectives.html>

This Lecture

- Diffusion Models
- Flow Matching